

```
0001 // 文件: apps/client/src/ShareApp.tsx
0002 import { Route, Routes } from "react-router-dom";
0003 import { getShareLegacyRouteMode } from "@lib/config.ts";
0004 import ShareNotFound from "@features/share/components/share-not-found.tsx";
0005 import SharedPage from "@pages/share/shared-page.tsx";
0006 import ShareLayout from "@features/share/components/share-layout.tsx";
0007 import ShareRedirect from "@pages/share/share-redirect.tsx";
0008
0009 export default function ShareApp() {
0010   const shareLegacyRouteMode = getShareLegacyRouteMode();
0011
0012   return (
0013     <Routes>
0014       <Route element={<ShareLayout />}>
0015         <Route path={"/share/:shareId/:pageSlug"} element={<SharedPage />} />
0016         <Route
0017           path={"/share/:shareId/p/:pageSlug"}
0018           element={<SharedPage />}
0019         />
0020         {shareLegacyRouteMode !== "removed" && (
0021           <Route path={"/share/p/:pageSlug"} element={<SharedPage />} />
0022         )}
0023       </Route>
0024       <Route path={"/share/:shareId"} element={<ShareRedirect />} />
0025       <Route path="*" element={<ShareNotFound />} />
0026     </Routes>
0027   );
0028 }
0029
0030 // 文件: apps/client/src/bootstrap-app.tsx
0031 import "@mantine/core/styles.css";
0032 import "@mantine/spotlight/styles.css";
0033 import "@mantine/notifications/styles.css";
0034 import "@mantine/dates/styles.css";
0035 import "@mantine/ui-refresh.css";
0036 import "@mantine/theme-palettes.css";
0037
0038 import ReactDOM from "react-dom/client";
0039 import { ErrorBoundary } from "react-error-boundary";
0040 import { MantineProvider } from "@mantine/core";
0041 import { BrowserRouter } from "react-router-dom";
0042 import { ModalsProvider } from "@mantine/modals";
0043 import { Notifications } from "@mantine/notifications";
0044 import { QueryClientProvider } from "@tanstack/react-query";
0045 import { HelmetProvider } from "react-helmet-async";
0046 import "i18n";
0047 import { PostHogProvider } from "posthog-js/react";
0048 import {
0049   getPostHogHost,
```

```
0051   getPostHogKey,  
0052   isCloud,  
0053   isPostHogEnabled,  
0054 } from "@lib/config.ts";  
0055 import posthog from "posthog-js";  
0056 import App from "../App.tsx";  
0057 import { mantineCssResolver, theme } from "@theme";  
0058 import {  
0059   getStoredPalette,  
0060   syncPaletteToDocument,  
0061 } from "@features/theme/theme-palette.ts";  
0062 import { ThemePaletteProvider } from "@features/theme/theme-palette-provider.tsx";  
0063 import { queryClient } from "@query-client.ts";  
0064  
0065 syncPaletteToDocument(getStoredPalette());  
0066  
0067 try {  
0068   if (isCloud() && isPostHogEnabled && getPostHogKey() && getPostHogHost()) {  
0069     posthog.init(getPostHogKey(), {  
0070       api_host: getPostHogHost(),  
0071       defaults: "2025-05-24",  
0072       disable_session_recording: true,  
0073       capture_pageleave: false,  
0074     });  
0075   }  
0076 } catch {  
0077   // ignore posthog init errors to avoid white screen  
0078 }  
0079  
0080 function AppErrorFallback({  
0081   error: _error,  
0082   resetErrorBoundary,  
0083 }): {  
0084   error: Error;  
0085   resetErrorBoundary: () => void;  
0086 }) {  
0087   return (  
0088     <div  
0089       style={{  
0090         display: "flex",  
0091         flexDirection: "column",  
0092         alignItems: "center",  
0093         justifyContent: "center",  
0094         minHeight: "100vh",  
0095         padding: 24,  
0096         background: "#f6f7f9",  
0097         color: "#495057",  
0098         fontFamily: "system-ui, sans-serif",  
0099         textAlign: "center",  
0100       }}  
0101     )
```

```
0101 >
0102 <p style={{ marginBottom: 16 }}>页面加载出错</p>
0103 <button
0104   type="button"
0105   onClick={() => {
0106     resetErrorBoundary();
0107     window.location.reload();
0108   }}
0109   style={{
0110     padding: "8px 16px",
0111     cursor: "pointer",
0112     background: "#228be6",
0113     color: "ffff",
0114     border: "none",
0115     borderRadius: 4,
0116   }}
0117 >
0118   重新加载
0119 </button>
0120 </div>
0121 );
0122 }
0123
0124 const rootEl = document.getElementById("root");
0125 if (!rootEl) {
0126   throw new Error("Root element #root not found");
0127 }
0128
0129 ReactDOM.createRoot(rootEl).render(
0130   <BrowserRouter>
0131     <MantineProvider theme={theme} cssVariablesResolver={mantineCssResolver}>
0132       <ModalProvider>
0133         <QueryClientProvider client={queryClient}>
0134           <Notifications position="bottom-center" limit={3} zIndex={10000} />
0135           <HelmetProvider>
0136             <PostHogProvider client={posthog}>
0137               <ErrorBoundary
0138                 fallbackComponent={AppErrorFallback}
0139                 onReset={() => {}}
0140               >
0141                 <ThemePaletteProvider>
0142                   <App />
0143                 </ThemePaletteProvider>
0144               </ErrorBoundary>
0145             </PostHogProvider>
0146           </HelmetProvider>
0147         </QueryClientProvider>
0148       </ModalProvider>
0149     </MantineProvider>
0150   </BrowserRouter>,

```

```
0151   });
0152
0153   // 文件: apps/client/src/bootstrap-share.tsx
0154   import './styles/ui-refresh.css';
0155   import './styles/share-theme.css';
0156
0157   import ReactDOM from 'react-dom/client';
0158   import { BrowserRouter } from 'react-router-dom';
0159   import { QueryClientProvider } from '@tanstack/react-query';
0160   import ShareApp from './ShareApp.tsx';
0161   import { applyShareReaderTheme } from '@features/share/share-color-scheme.ts';
0162   import { ShareErrorBoundary } from '@features/share/components/share-error-boundary.tsx';
0163   import { useShareTranslation } from '@features/share/share-translations.ts';
0164   import { queryClient } from '@query-client.ts';
0165
0166   applyShareReaderTheme();
0167
0168   function ShareErrorFallback({
0169     error,
0170     resetErrorBoundary,
0171   }: {
0172     error: Error;
0173     resetErrorBoundary: () => void;
0174   }) {
0175     const { t } = useShareTranslation();
0176     const errorSummary = [error.name, error.message]
0177       .filter(Boolean)
0178       .join(' ');
0179     .slice(0, 240);
0180
0181     return (
0182       <div
0183         style={{
0184           display: 'flex',
0185           flexDirection: 'column',
0186           alignItems: 'center',
0187           justifyContent: 'center',
0188           minHeight: '100vh',
0189           padding: 24,
0190           background: '#f6f7f9',
0191           color: '#495057',
0192           fontFamily: 'system-ui, sans-serif',
0193           textAlign: 'center',
0194         }}
0195     >
0196     <p style={{ marginBottom: 16 }}><t("Page failed to load")></p>
0197     {errorSummary} && (
0198       <p
0199         style={{
0200           margin: '0 0 16px',
```

```
0201         maxWidth: 560,
0202         color: "#6c757d",
0203         fontSize: 13,
0204         lineHeight: 1.5,
0205         wordBreak: "break-word",
0206       }}
0207     >
0208     {errorSummary}
0209   </p>
0210 )}
0211 <button
0212   type="button"
0213   onClick={() => {
0214     resetErrorBoundary();
0215     window.location.reload();
0216   }}
0217   style={{
0218     padding: "8px 16px",
0219     cursor: "pointer",
0220     background: "#228be6",
0221     color: "#fff",
0222     border: "none",
0223     borderRadius: 4,
0224   }}
0225   >
0226   {t("Reload")}
0227 </button>
0228 </div>
0229 );
0230 }
0231
0232 const rootEl = document.getElementById("root");
0233 if (!rootEl) {
0234   throw new Error("Root element #root not found");
0235 }
0236
0237 rootEl.setAttribute("data-share-bootstrap-mounted", "true");
0238 if (typeof (window as any).__shareDebug === "function") {
0239   (window as any).__shareDebug("Bootstrap: React mounting");
0240 }
0241
0242 ReactDOM.createRoot(rootEl).render(
0243   <BrowserRouter>
0244     <QueryClientProvider client={queryClient}>
0245       <ShareErrorBoundary
0246         renderFallback={props => <ShareErrorFallback {...props} />}
0247       >
0248         <ShareApp />
0249       </ShareErrorBoundary>
0250     </QueryClientProvider>
```

```
0251   </BrowserRouter>,
0252   );
0253
0254   // 文件: apps/client/src/components/common/avatar-uploader.tsx
0255   import React, { useRef } from "react";
0256   import { Menu, Box, Loader } from "@mantine/core";
0257   import { useTranslation } from "react-i18next";
0258   import { IconTrash, IconUpload } from "tabler/icons-react";
0259   import { CustomAvatar } from "@components/ui/custom-avatar.tsx";
0260   import { AvatarIconType } from "@features/attachments/types/attachment.types.ts";
0261   import { notifications } from "@mantine/notifications";
0262
0263   interface AvatarUploaderProps {
0264     currentImageUrl?: string | null;
0265     fallbackName?: string;
0266     radius?: string | number;
0267     size?: string | number;
0268     variant?: string;
0269     type: AvatarIconType;
0270     onUpload: (file: File) => Promise<void>;
0271     onRemove: () => Promise<void>;
0272     isLoading?: boolean;
0273     disabled?: boolean;
0274   }
0275
0276   export default function AvatarUploader({
0277     currentImageUrl,
0278     fallbackName,
0279     radius,
0280     variant,
0281     size,
0282     type,
0283     onUpload,
0284     onRemove,
0285     isLoading = false,
0286     disabled = false,
0287   }: AvatarUploaderProps) {
0288     const { t } = useTranslation();
0289     const fileInputRef = useRef<HTMLInputElement>(null);
0290
0291     const handleFileInputChange = async (
0292       event: React.ChangeEvent<HTMLInputElement>,
0293     ) => {
0294       const file = event.target.files?.[0];
0295       if (!file || disabled) {
0296         return;
0297       }
0298
0299       // Validate file size (max 10MB)
0300       const maxSizeInBytes = 10 * 1024 * 1024;
```

```
0301   if (file.size > maxSizeInBytes) {
0302     notifications.show({
0303       message: t("Image exceeds 10MB limit."),
0304       color: "red",
0305     });
0306     // Reset the input
0307     if (fileInputRef.current) {
0308       fileInputRef.current.value = "";
0309     }
0310     return;
0311   }
0312
0313   try {
0314     await onUpload(file);
0315   } catch (error) {
0316     console.error(error);
0317     notifications.show({
0318       message: t("Failed to upload image"),
0319       color: "red",
0320     });
0321   }
0322
0323   // Reset the input so the same file can be selected again
0324   if (fileInputRef.current) {
0325     fileInputRef.current.value = "";
0326   }
0327 };
0328
0329 const handleUploadClick = () => {
0330   if (fileInputRef.current) {
0331     fileInputRef.current.click();
0332   } else {
0333     console.error("File input ref is null!");
0334   }
0335 };
0336
0337 const handleRemove = async () => {
0338   if (disabled) return;
0339
0340   try {
0341     await onRemove();
0342     notifications.show({
0343       message: t("Image removed successfully"),
0344     });
0345   } catch (error) {
0346     console.error(error);
0347     notifications.show({
0348       message: t("Failed to remove image"),
0349       color: "red",
0350     });
0351   }
```

```
0351     }
0352   };
0353
0354   return (
0355     <Box>
0356       <Input
0357         type="file"
0358         ref={fileInputRef}
0359         onChange={handleFileInputChange}
0360         accept="image/png,image/jpeg,image/jpg"
0361         style={{ display: "none" }}
0362       />
0363
0364     <Menu shadow="md" width={200} withArrow disabled={disabled || isLoading}>
0365       <Menu.Target>
0366         <Box style={{ position: "relative", display: "inline-block" }}>
0367           <CustomAvatar
0368             component="button"
0369             size={size}
0370             avatarUrl={currentImageUrl}
0371             name={fallbackName}
0372             style={{
0373               cursor: disabled || isLoading ? "default" : "pointer",
0374               opacity: isLoading ? 0.6 : 1,
0375             }}
0376             radius={radius}
0377             variant={variant}
0378             type={type}
0379           />
0380           {isLoading && (
0381             <Box
0382               style={{
0383                 position: "absolute",
0384                 top: "50%",
0385                 left: "50%",
0386                 transform: "translate(-50%, -50%)",
0387                 zIndex: 200,
0388               }}
0389             >
0390               <Loader size="sm" />
0391             </Box>
0392           )}
0393         </Box>
0394       </Menu.Target>
0395
0396       <Menu.Dropdown>
0397         <Menu.Item
0398           leftSection={<IconUpload size={16} />}
0399           disabled={isLoading || disabled}
0400           onClick={handleUploadClick}
0401         />
0402       </Menu.Dropdown>
0403     </Menu>
0404   );
0405 }
```

```
0401         >
0402         {t("Upload image")}
0403     </Menu.Item>
0404
0405     {currentImageUrl && (
0406     <Menu.Item
0407         leftSection={<IconTrash size={16} />}
0408         color="red"
0409         onClick={handleRemove}
0410         disabled={isLoading || disabled}
0411     >
0412         {t("Remove image")}
0413     </Menu.Item>
0414     )}
0415 </Menu.Dropdown>
0416 </Menu>
0417 </Box>
0418 );
0419 }
0420
0421 // 文件: apps/client/src/components/common/copy-button.tsx
0422 // Source: https://github.com/mantinedev/mantine/blob/master/packages/@mantine/core/src/components/CopyButton/CopyButton.tsx - MIT
0423 // modified to use the polyfilled clipboard api
0424 import React from "react";
0425 import { useClipboard } from "@hooks/use-clipboard";
0426 import { useProps } from "@mantine/core";
0427
0428 interface CopyButtonProps {
0429     /** Children callback, provides current status and copy function as an argument */
0430     children: (payload: { copied: boolean; copy: () => void }) => React.ReactNode;
0431
0432     /** Value that is copied to the clipboard when the button is clicked */
0433     value: string;
0434
0435     /** Copied status timeout in ms @default `1000` */
0436     timeout?: number;
0437 }
0438
0439 const defaultProps = {
0440     timeout: 1000,
0441 } satisfies Partial<CopyButtonProps>;
0442
0443 export function CopyButton(props: CopyButtonProps) {
0444     const { children, timeout, value, ...others } = useProps(
0445         "CopyButton",
0446         defaultProps,
0447         props,
0448     );
0449     const clipboard = useClipboard({ timeout });
0450     const copy = () => clipboard.copy(value);
```

```
0451     return <{children({ copy, copied: clipboard.copied, ...others })}</>;
0452   }
0453
0454   CopyButton.displayName = "@mantine/core/CopyButton";
0455
0456   // 文件: apps/client/src/components/common/copy.tsx
0457   import { ActionIcon, Tooltip } from "@mantine/core";
0458   import { CopyButton } from "@components/common/copy-button";
0459   import { IconCheck, IconCopy } from "@tabler/icons-react";
0460   import React from "react";
0461   import { useTranslation } from "react-i18next";
0462
0463   interface CopyProps {
0464     text: string;
0465   }
0466   export default function CopyTextButton({ text }: CopyProps) {
0467     const { t } = useTranslation();
0468
0469     return (
0470       <CopyButton value={text} timeout={2000}>
0471         {{{ copied, copy }} => (
0472           <Tooltip
0473             label={copied ? t("Copied") : t("Copy")}
0474             withArrow
0475             position="right"
0476           >
0477             <ActionIcon
0478               color={copied ? "teal" : "gray"}
0479               variant="subtle"
0480               onClick={copy}
0481             >
0482               {copied ? <IconCheck size={16} /> : <IconCopy size={16} />}
0483             </ActionIcon>
0484           </Tooltip>
0485         )}
0486       </CopyButton>
0487     );
0488   }
0489
0490   // 文件: apps/client/src/components/common/export-modal.tsx
0491   import {
0492     Modal,
0493     Button,
0494     Group,
0495     Text,
0496     Select,
0497     Switch,
0498     Divider,
0499   } from "@mantine/core";
0500   import { exportPage } from "@features/page/services/page-service.ts";
```

```
0501 import { useState } from "react";
0502 import { ExportFormat } from "@features/page/types/page.types.ts";
0503 import { notifications } from "@mantine/notifications";
0504 import { exportSpace } from "@features/space/services/space-service";
0505 import { useTranslation } from "react-i18next";
0506
0507 interface ExportModalProps {
0508   id: string;
0509   type: "space" | "page";
0510   open: boolean;
0511   onClose: () => void;
0512 }
0513
0514 export default function ExportModal({
0515   id,
0516   type,
0517   open,
0518   onClose,
0519 }: ExportModalProps) {
0520   const [format, setFormat] = useState<ExportFormat>(ExportFormat.Markdown);
0521   const [includeChildren, setIncludeChildren] = useState<boolean>(false);
0522   const [includeAttachments, setIncludeAttachments] = useState<boolean>(false);
0523   const [isExporting, setIsExporting] = useState<boolean>(false);
0524   const { t } = useTranslation();
0525
0526   const handleExport = async () => {
0527     setIsExporting(true);
0528     try {
0529       if (type === "page") {
0530         await exportPage({
0531           pageId: id,
0532           format,
0533           includeChildren,
0534           includeAttachments,
0535         });
0536       }
0537       if (type === "space") {
0538         await exportSpace({ spaceId: id, format, includeAttachments });
0539       }
0540       notifications.show({
0541         message: t("Export successful"),
0542       });
0543       onClose();
0544     } catch (err) {
0545       notifications.show({
0546         message: "Export failed:" + err.response?.data.message,
0547         color: "red",
0548       });
0549       console.error("export error", err);
0550     } finally {
0551     }
```

```
0551         setIsExporting(false);
0552     }
0553 };
0554
0555 const handleChange = (format: ExportFormat) => {
0556     setFormat(format);
0557 };
0558
0559 return (
0560     <Modal.Root
0561         opened={open}
0562         onClose={onClose}
0563         size={500}
0564         padding="xl"
0565         yOffset="10vh"
0566         xOffset={0}
0567         mah={400}
0568         onClick={(e) => e.stopPropagation()}
0569     >
0570     <Modal.Overlay blur={1} />
0571     <Modal.Content style={{ overflow: "hidden" }}>
0572         <Modal.Header py={0}>
0573             <Modal.Title fw={500}>{t(`Export ${type}`)}</Modal.Title>
0574             <Modal.CloseButton />
0575         </Modal.Header>
0576         <Modal.Body>
0577             <Group justify="space-between" wrap="nowrap">
0578                 <div>
0579                     <Text size="md">{t("Format")}</Text>
0580                 </div>
0581                 <ExportFormatSelection format={format} onChange={handleChange} />
0582             </Group>
0583
0584             {type === "page" && (
0585                 <div>
0586                     <Divider my="sm" />
0587
0588                     <Group justify="space-between" wrap="nowrap">
0589                         <div>
0590                             <Text size="md">{t("Include subpages")}</Text>
0591                         </div>
0592                         <Switch
0593                             onChange={event =>
0594                                 setIncludeChildren(event.currentTarget.checked)
0595                             }
0596                             checked={includeChildren}
0597                         />
0598                     </Group>
0599
0600                     <Group justify="space-between" wrap="nowrap" mt="md">
```

```
0601         <div>
0602             <Text size="md">{t("Include attachments")}</Text>
0603         </div>
0604         <Switch
0605             onChange={event =>
0606                 setIncludeAttachments(event.currentTarget.checked)
0607             }
0608             checked={includeAttachments}
0609         />
0610     </Group>
0611 </>
0612 )}
0613
0614 {type === "space" && (
0615     <
0616         <Divider my="sm" />
0617
0618         <Group justify="space-between" wrap="nowrap">
0619             <div>
0620                 <Text size="md">{t("Include attachments")}</Text>
0621             </div>
0622             <Switch
0623                 onChange={event =>
0624                     setIncludeAttachments(event.currentTarget.checked)
0625                 }
0626                 checked={includeAttachments}
0627             />
0628         </Group>
0629     </>
0630 )}
0631
0632 <Group justify="end" mt="md" gap="xs">
0633     <Button onClick={onClose} variant="subtle">
0634         {t("Cancel")}
0635     </Button>
0636     <Button onClick={handleExport} loading={isExporting}>{t("Export")}</Button>
0637 </Group>
0638 </Modal.Body>
0639 </Modal.Content>
0640 </Modal.Root>
0641 };
0642 }
0643
0644 interface ExportFormatSelection {
0645     format: ExportFormat;
0646     onChange: (value: string) => void;
0647 }
0648 function ExportFormatSelection({ format, onChange }: ExportFormatSelection) {
0649     const { t } = useTranslation();
0650 }
```

```
0651     return (
0652       <Select
0653         data={[
0654           { value: "markdown", label: "Markdown" },
0655           { value: "html", label: "HTML" },
0656         ]}
0657         defaultValue={format}
0658         onChange={onChange}
0659         styles={{ wrapper: { maxWidth: 120 } }}
0660         comboboxProps={{ width: "120" }}
0661         allowDeselect={false}
0662         withCheckIcon={false}
0663         aria-label={t("Select export format")}
0664       />
0665     );
0666   }
0667
0668   // 文件: apps/client/src/components/common/no-table-results.tsx
0669   import { Table, Text } from "@mantine/core";
0670   import React from "react";
0671   import { useTranslation } from "react-i18next";
0672
0673   interface NoTableResultsProps {
0674     colSpan: number;
0675     text?: string;
0676   }
0677
0678   export default function NoTableResults({ colSpan, text }: NoTableResultsProps) {
0679     const { t } = useTranslation();
0680     return (
0681       <Table.Tr>
0682         <Table.Td colSpan={colSpan}>
0683           <Text fw={500} c="dimmed" ta="center">
0684             {text || t("No results found...")}
0685           </Text>
0686         </Table.Td>
0687       </Table.Tr>
0688     );
0689   }
0690
0691   // 文件: apps/client/src/components/common/paginate.tsx
0692   import { Button, Group } from "@mantine/core";
0693   import { useTranslation } from "react-i18next";
0694
0695   export interface PagePaginationProps {
0696     hasPrevPage: boolean;
0697     hasNextPage: boolean;
0698     onPrev: () => void;
0699     onNext: () => void;
0700   }
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 15 页
0701	export default function Paginate({	
0702	hasPrevPage,	
0703	hasNextPage,	
0704	onPrev,	
0705	onNext,	
0706	): PagePaginationProps) {	
0707	const { t } = useTranslation();	
0708		
0709	if (!hasPrevPage && !hasNextPage) {	
0710	return null;	
0711	}	
0712		
0713	return (	
0714	<Group mt="md" justify="flex-end">	
0715	<Button	
0716	variant="default"	
0717	size="compact-sm"	
0718	onClick={onPrev}	
0719	disabled={!hasPrevPage}	
0720	>	
0721	{t("Prev")}	
0722	</Button>	
0723		
0724	<Button	
0725	variant="default"	
0726	size="compact-sm"	
0727	onClick={onNext}	
0728	disabled={!hasNextPage}	
0729	>	
0730	{t("Next")}	
0731	</Button>	
0732	</Group>	
0733	);	
0734	}	
0735		
0736	// 文件: apps/client/src/components/common/recent-changes.tsx	
0737	import {	
0738	Text,	
0739	Group,	
0740	UnstyledButton,	
0741	Badge,	
0742	Table,	
0743	ActionIcon,	
0744	} from "@mantine/core";	
0745	import { Link } from "react-router-dom";	
0746	import PageListSkeleton from "@components/ui/page-list-skeleton.tsx";	
0747	import { buildPageUrl } from "@features/page/page.utils.ts";	
0748	import { formattedDate } from "@lib/time.ts";	
0749	import {	
0750	prefetchPage,	

```
0751   useRecentChangesQuery,
0752 } from "@features/page/queries/page-query.ts";
0753 import { IconFileDescription, IconFiles } from "@tabler/icons-react";
0754 import { EmptyState } from "@components/ui/empty-state.tsx";
0755 import { getSpaceUrl } from "@lib/config.ts";
0756 import { useTranslation } from "react-i18next";
0757 import { getInitialsColor } from "@lib/get-initials-color.ts";
0758 import type { IPage } from "@features/page/types/page.types.ts";
0759
0760 interface Props {
0761   spaceId?: string;
0762 }
0763
0764 const preloadPageRoute = () => import("@pages/page/page");
0765
0766 function prefetchRecentPage(page: Pick<IPage, "slugId">) {
0767   void preloadPageRoute();
0768   void prefetchPage({ pageId: page.slugId });
0769 }
0770
0771 export default function RecentChanges({ spaceId }: Props) {
0772   const { t } = useTranslation();
0773   const { data: pages, isLoading, isError } = useRecentChangesQuery(spaceId);
0774
0775   if (isLoading) {
0776     return <PageListSkeleton />;
0777   }
0778
0779   if (isError) {
0780     return <Text>{t("Failed to fetch recent pages")}</Text>;
0781   }
0782
0783   return pages && pages.items.length > 0 ? (
0784     <Table.ScrollContainer minWidth={500}>
0785       <Table highlightOnHover verticalSpacing="sm">
0786         <Table.Tbody>
0787           {pages.items.map((page) => (
0788             <Table.Tr key={page.id}>
0789               <Table.Td>
0790                 <UnstyledButton
0791                   component={Link}
0792                   to={buildPageUrl({ page?.space.slug, page.slugId, page.title })}
0793                   onFocus={() => prefetchRecentPage(page)}
0794                   onPointerDown={() => prefetchRecentPage(page)}
0795                   onPointerEnter={preloadPageRoute}
0796                 >
0797                   <Group wrap="nowrap">
0798                     {page.icon || (
0799                       <ActionIcon variant="transparent" color="gray" size={18}>
0800                         <IconFileDescription size={18} />
0801                     )}
0802                   <PageTitle>{page.title}</PageTitle>
0803                   <PageDate>{page.createdAt}</PageDate>
0804                 </Group>
0805               </Table.Td>
0806             </Table.Tr>
0807           ))}
0808         </Table.Tbody>
0809       </Table>
0810     </Table.ScrollContainer>
0811   ) : <EmptyState />;
0812 }
```

```
0801         </ActionIcon>
0802     })
0803
0804     <Text fw={500} size="md" lineClamp={1}>
0805       {page.title || t("Untitled")}
0806     </Text>
0807   </Group>
0808 </UnstyledButton>
0809 </Table.Td>
0810 {!spaceId && (
0811   <Table.Td>
0812     <Badge
0813       color={getInitialsColor(page?.space.name)}
0814       variant="light"
0815       component={Link}
0816       to={getSpaceUrl(page?.space.slug)}
0817       style={{ cursor: "pointer" }}
0818     >
0819       {page?.space.name}
0820     </Badge>
0821   </Table.Td>
0822 )}
0823 <Table.Td>
0824   <Text
0825     c="dimmed"
0826     style={{ whiteSpace: "nowrap" }}
0827     size="xs"
0828     fw={500}
0829   >
0830     {formattedDate(page.updatedAt)}
0831   </Text>
0832 </Table.Td>
0833 </Table.Tr>
0834   )))
0835 </Table.Tbody>
0836 </Table>
0837 </Table.ScrollContainer>
0838 ) : (
0839   <EmptyState
0840     icon={IconFiles}
0841     title={t("No pages yet")}
0842     description={t("Pages you create will show up here.")}
0843   />
0844 );
0845 }
0846
0847 // 文件: apps/client/src/components/common/route-transition.tsx
0848 import React, { useEffect, useState } from "react";
0849 import { useLocation } from "react-router-dom";
0850
```

```
0851 /**
0852  * Wraps children with a subtle fade-in transition on route change.
0853  * Place inside <Routes> or around the main content area.
0854  */
0855 export function RouteTransition({ children }: { children: React.ReactNode }) {
0856   const location = useLocation();
0857   const [displayChildren, setDisplayChildren] = useState(children);
0858   const [transitionKey, setTransitionKey] = useState(location.pathname);
0859   const [isTransitioning, setIsTransitioning] = useState(false);
0860
0861   useEffect(() => {
0862     if (location.pathname !== transitionKey) {
0863       setIsTransitioning(true);
0864       const timeout = setTimeout(() => {
0865         setDisplayChildren(children);
0866         setTransitionKey(location.pathname);
0867         // Trigger fade-in on next frame
0868         requestAnimationFrame(() => {
0869           setIsTransitioning(false);
0870         });
0871       }, 80); // Brief fade-out duration
0872
0873       return () => clearTimeout(timeout);
0874     } else {
0875       setDisplayChildren(children);
0876     }
0877   }, [children, location.pathname, transitionKey]);
0878
0879   return (
0880     <div
0881       style={{
0882         opacity: isTransitioning ? 0 : 1,
0883         transition: "opacity 120ms ease-in-out",
0884         minHeight: "100%",
0885       }}
0886     >
0887       {displayChildren}
0888     </div>
0889   );
0890 }
0891
0892 /* 文件: apps/client/src/components/common/search-input.module.css */
0893 .wrapper {
0894   width: 100%;
0895 }
0896
0897 .input {
0898   width: 100%;
0899 }
0900
```

```
0901 // 文件: apps/client/src/components/common/search-input.tsx
0902 import React, { useState, useEffect } from "react";
0903 import { TextInput, Group } from "@mantine/core";
0904 import { useDebounceValue } from "@mantine/hooks";
0905 import { IconSearch } from "@tabler/icons-react";
0906 import { useTranslation } from "react-i18next";
0907 import classes from "./search-input.module.css";
0908
0909 export interface SearchInputProps {
0910   placeholder?: string;
0911   debounceDelay?: number;
0912   onSearch: (value: string) => void;
0913 }
0914
0915 export function SearchInput({
0916   placeholder,
0917   debounceDelay = 500,
0918   onSearch,
0919 }: SearchInputProps) {
0920   const { t } = useTranslation();
0921   const [value, setValue] = useState("");
0922   const [debouncedValue] = useDebounceValue(value, debounceDelay);
0923
0924   useEffect(() => {
0925     onSearch(debouncedValue);
0926   }, [debouncedValue, onSearch]);
0927
0928   return (
0929     <Group mb="sm" className={classes.wrapper}>
0930       <TextInput
0931         size="sm"
0932         placeholder={placeholder || t("Search...")}
0933         leftSection=<IconSearch size={15} stroke={1.75} />
0934         radius="md"
0935         className={classes.input}
0936         value={value}
0937         onChange={(e) => setValue(e.currentTarget.value)}
0938       />
0939     </Group>
0940   );
0941 }
0942
0943 // 文件: apps/client/src/components/common/user-info.tsx
0944 import { Group, Text } from "@mantine/core";
0945 import { CustomAvatar } from "@components/ui/custom-avatar.tsx";
0946 import React from "react";
0947 import { IUser } from "@features/user/types/user.types.ts";
0948
0949 interface UserInfoProps {
0950   user: Partial<IUser>;
```

```
0951   size?: string;
0952 }
0953 export function UserInfo({ user, size }: UserInfoProps) {
0954   return (
0955     <Group gap="sm" wrap="nowrap">
0956       <CustomAvatar avatarUrl={user?.avatarUrl} name={user?.name} size={size} />
0957       <div>
0958         <Text fz="sm" fw={500} lineClamp={1}>
0959           {user?.name}
0960         </Text>
0961         <Text fz="xs" c="dimmed">
0962           {user?.email}
0963         </Text>
0964       </div>
0965     </Group>
0966   );
0967 }
0968
0969 // 文件: apps/client/src/components/icons/airtable-icon.tsx
0970 import { rem } from '@mantine/core';
0971
0972 interface Props {
0973   size?: number | string;
0974 }
0975
0976 export function AirtableIcon({ size }: Props) {
0977   return (
0978     <svg
0979       xmlns="http://www.w3.org/2000/svg"
0980       viewBox="0 0 256 215"
0981       style={{ width: rem(size), height: rem(size) }}
0982     >
0983       <path
0984         fill="#ffb000"
0985         d="M114.259 2.701 18.86 42.176c-5.305 2.195-5.25 9.73 0.89 11.847l95.797 37.989a35.544 35.544 0 0 0 26.208 0l95.799-37.99c5.337-2.115 5.393-9.65 0.86-11.846l141.442 2.7a35.549 35.549 0 0 0-27.183 0"
0986       />
0987       <path
0988         fill="#26b5f8"
0989         d="M136.35 112.757v94.902c0 4.514 4.55 7.605 8.746 5.942l106.748-41.435a6.39 6.39 0 0 0 4.035-5.941v71.322c0-4.514-4.551-7.604-8.747-5.941l-106.748 41.434a6.392 6.392 0 0 0-4.035 5.942"
0990       />
0991       <path
0992         fill="#ed3049"
0993         d="m111.423 117.654-31.68 15.296-3.217 1.555l9.65 166.548c5.411 168.593 0 165.504 0 160.795v71.72c0-1.704 874-3.175 2.046-4.283a7.266 7.266 0 0 1 1.618-1.213c1.598-.959 3.878-1.215 5.816-.448l101.41 40.18c5.155 2.045 5.56 9
0994       />
0995       <path
0996         fillOpacity={0.25}
0997         d="m111.423 117.654-31.68 15.296l2.045 67.438a7.266 7.266 0 0 1 1.618-1.213c1.598-.959 3.878-1.215 5.816-.448l101.41 40.18c5.155 2.045 5.56 9.268 533 11.697"
0998       />
0999     </svg>
1000   );

```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料

鲜知（北京）科技有限公司 | 第 21 页

1001

}

1002

1003

// 文件: apps/client/src/components/icons/confluence-icon.tsx

1004

import { rem } from '@mantine/core';

1005

1006

interface Props {

1007

size?: number | string;

1008

}

1009

1010

export function ConfluenceIcon({ size }: Props) {

1011

return (

1012

<svg

1013

xmlns="http://www.w3.org/2000/svg"

1014

viewBox="0 0 24 24"

1015

fill="none"

1016

stroke="currentColor"

1017

strokeWidth="2"

1018

style={{ width: rem(size), height: rem(size) }}

1019

>

1020

<path d="M.87 18.257c-.248.382-.53.875-.763 1.245a.764.764 0 0 0 .255 1.041.965 3.054a.764.764 0 0 0 1.058-.26c.199-.332.454-.763.733-1.221 1.967-3.247 3.945-2.853 7.508-1.1461.957 2.337a.764 0 0 0 1.028-.38212.364-5.34

1021

</svg>

1022

);

1023

}

1024

1025

// 文件: apps/client/src/components/icons/figma-icon.tsx

1026

import { rem } from '@mantine/core';

1027

1028

interface Props {

1029

size?: number | string;

1030

}

1031

1032

export function FigmaIcon({ size }: Props) {

1033

return (

1034

<svg

1035

xmlns="http://www.w3.org/2000/svg"

1036

viewBox="0 0 24 24"

1037

style={{ width: rem(size), height: rem(size) }}

1038

>

1039

<g fill="none" fillRule="evenodd" transform="translate(4)">

1040

<circle cx={12} cy={12} r={4} fill="#19bcfe" />

1041

<path fill="#9cf83" d="M4 24a4 4 0 0 4 4v-4h4a4 4 0 1 0 0 8z" />

1042

<path fill="#a259ff" d="M4 16h4v4a4 4 0 1 0 0 8z" />

1043

<path fill="#f24e1e" d="M4 8h4v4a4 4 0 1 0 0 8z" />

1044

<path fill="#ff7262" d="M12 8h8v4a4 4 0 1 1 0 8z" />

1045

</g>

1046

</svg>

1047

);

1048

}

1049

1050

// 文件: apps/client/src/components/icons/framer-icon.tsx

```
1051 import { rem } from '@mantine/core';
1052
1053 interface Props {
1054   size?: number | string;
1055 }
1056
1057 export function FramerIcon({ size }: Props) {
1058   return (
1059     <svg
1060       xmlns="http://www.w3.org/2000/svg"
1061       viewBox="0 0 24 24"
1062       style={{ width: rem(size), height: rem(size) }}
1063     >
1064       <path d="M4 0h16v8h-8z" />
1065     </svg>
1066   );
1067 }
1068
1069 // 文件: apps/client/src/components/icons/google-drive-icon.tsx
1070 import { rem } from '@mantine/core';
1071
1072 interface Props {
1073   size?: number | string;
1074 }
1075
1076 export function GoogleDriveIcon({ size }: Props) {
1077   return (
1078     <svg
1079       xmlns="http://www.w3.org/2000/svg"
1080       viewBox="0 0 87.3 78"
1081       style={{ width: rem(size), height: rem(size) }}
1082     >
1083       <path d="m6.6 66.85 3.85 6.65c.8 1.4 1.95 2.5 3.3 3.3l13.75-23.8h-27.5c0 1.55.4 3.1 1.2 4.5z" fill="#0066da" />
1084       <path d="m43.65 25-13.75-23.8c-1.35-.8-2.5 1.9-3.3 3.3l-25.4 44a9.06 9.06 0 0 -1.2 4.5h27.5z" fill="#00ac47" />
1085       <path d="m73.55 76.8c1.35-.8 2.5-1.9 3.3-3.3l11.6-2.75 7.65-13.25c.8-1.4 1.2-2.95 1.2-4.5h-27.502l5.852 11.5z"
1086         fill="#ea4335" />
1087       <path d="m43.65 25 13.75-23.8c-1.35-.8-2.9-1.2-4.5-1.2h-18.5c-1.6 0-3.15.45-4.5 1.2z" fill="#00832d" />
1088       <path d="m59.8 53h-32.3l-13.75 23.8c1.35.8 2.9 1.2 4.5 1.2h50.8c1.6 0 3.15-.45 4.5-1.2z" fill="#2684fc" />
1089       <path d="m73.4 26.5-12.7-22c-.8-1.4-1.95-2.5-3.3-3.3l-13.75 23.8 16.15 28h27.45c0-1.55-.4-3.1-1.2-4.5z"
1090         fill="#ffb800" />
1091     </svg>
1092   );
1093 }
1094
1095 // 文件: apps/client/src/components/icons/google-icon.tsx
1096 import { rem } from "@mantine/core";
1097
1098 interface Props {
1099   size?: number | string;
1100 }
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料

鲜知（北京）科技有限公司 | 第 23 页

```
1101
1102 export function GoogleIcon({ size }: Props) {
1103   return (
1104     <svg
1105       xmlns="http://www.w3.org/2000/svg"
1106       preserveAspectRatio="xMidYMid"
1107       viewBox="0 0 256 262"
1108       style={{ width: rem(size), height: rem(size) }}
1109     >
1110       <path
1111         fill="#4285F4"
1112         d="M255.878 133.451c0-10.734-.871-18.567-2.756-26.69H130.55v48.448h71.947c-1.45 12.04-9.283 30.172-26.69 42.356l-.244 1.622 38.755 30.023 2.685,268c24.659-22.774 38.875-56.282 38.875-96.027"
1113       />
1114       <path
1115         fill="#34A853"
1116         d="M130.55 261.1c35.248 0 64.839-11.605 86.453-31.622l-41.196-31.913c-11.024 7.688-25.82 13.055-45.257 13.055-34.523 0-63.824-22.773-74.269-54.251-1.531,13-40.298 31.187-.527 1.465c35.393 231.798 79.49 261.1 130.55 261.1"
1117       />
1118       <path
1119         fill="#FBBC05"
1120         d="M56.281 156.37c-2.756-8.123-4.351-16.827-4.351-25.82 0-8.994 1.595-17.697 4.206-25.82l-.073-1.73l15.26 71.312l-1.335,635c5.077 89.644 0 109.517 0 130.55s5.077 40.905 13.925 58.602l42.356-32.782"
1121       />
1122       <path
1123         fill="#EB4335"
1124         d="M130.55 50.479c24.514 0 41.05 10.589 50.479 19.438l36.844-35.974c195.245 12.91 165.798 0 130.55 0 79.49 0 35.393 29.301 13.925 71.947l42.211 32.783c10.59-31.477 39.891-54.251 74.414-54.251"
1125       />
1126     </svg>
1127   );
1128 }
1129
1130 // 文件: apps/client/src/components/icons/google-sheets-icon.tsx
1131 import { rem } from '@mantine/core';
1132
1133 interface Props {
1134   size?: number | string;
1135 }
1136
1137 export function GoogleSheetsIcon({ size }: Props) {
1138   return (
1139     <svg
1140       xmlns="http://www.w3.org/2000/svg"
1141       viewBox="0 0 48 48"
1142       style={{ width: rem(size), height: rem(size) }}
1143     >
1144       <path fill="#43a047" d="M37,45H11c-1.657,0-3-1.343-3-3V6c0-1.657,1.343-3,3-3h19l10,10v29C40,43.657,38.657,45,37,45z"/>
1145       <path fill="#c8e6c9" d="M40 13L30 13 30 3z"/>
1146       <path fill="#2e7d32" d="M30 13L40 23 40 13z"/>
1147     <path
1148       fill="#e8f5e9"
1149       d="M31,23H17h-2v2v2v2v2v2v2h18v-2v-2v-2v-2v-2v-2v-2H31z M17,25h4v2h-4V25z M17,29h4v2h-4V29z M17,33h4v2h-4V33z M31,35h-8v-2h8V35z M31,31h-8v-2h8V31z M31,27h-8v-2h8V27z"
1150     />
1151   );
1152 }
```

file:///root/coderepository/docmost/docs/software-copyright/20260427_superchat_v1_0/generated/SuperChat_%E7%9F%A5%... 23/60

```
1151     </svg>
1152   );
1153 }
1154
1155 // 文件: apps/client/src/components/icons/icon-columns-4.tsx
1156 import { rem } from "@mantine/core";
1157
1158 type Props = {
1159   size?: number | string;
1160   stroke?: number;
1161 };
1162
1163 export function IconColumns4({ size = 24, stroke = 2 }: Props) {
1164   return (
1165     <svg
1166       xmlns="http://www.w3.org/2000/svg"
1167       width={rem(size)}
1168       height={rem(size)}
1169       viewBox="0 0 24 24"
1170       fill="none"
1171       stroke="currentColor"
1172       strokeWidth={stroke}
1173       strokeLinecap="round"
1174       strokeLinejoin="round"
1175     >
1176       <path d="M3 4a1 1 0 0 1 1 -1h16a1 1 0 0 1 1 1v16a1 1 0 0 1 -1 1h-16a1 1 0 0 1 -1 -1v-16" />
1177       <path d="M7.5 3v18" />
1178       <path d="M12 3v18" />
1179       <path d="M16.5 3v18" />
1180     </svg>
1181   );
1182 }
1183
1184 // 文件: apps/client/src/components/icons/icon-columns-5.tsx
1185 import { rem } from "@mantine/core";
1186
1187 type Props = {
1188   size?: number | string;
1189   stroke?: number;
1190 };
1191
1192 export function IconColumns5({ size = 24, stroke = 2 }: Props) {
1193   return (
1194     <svg
1195       xmlns="http://www.w3.org/2000/svg"
1196       width={rem(size)}
1197       height={rem(size)}
1198       viewBox="0 0 24 24"
1199       fill="none"
1200       stroke="currentColor"
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料	鲜知（北京）科技有限公司 第 25 页
1201 strokeWidth={stroke}	
1202 strokeLinecap="round"	
1203 strokeLinejoin="round"	
1204 >	
1205 <path d="M3 4a1 1 0 0 1 1 -.1h16a1 1 0 0 1 1 1v16a1 1 0 0 1 -1 1h-16a1 1 0 0 1 -1 -.1v-16" />	
1206 <path d="M6.6 3v18" />	
1207 <path d="M10.2 3v18" />	
1208 <path d="M13.8 3v18" />	
1209 <path d="M17.4 3v18" />	
1210 </svg>	
1211);	
1212 }	
1213	
1214 // 文件: apps/client/src/components/icons/icon-drawio.tsx	
1215 import { rem } from '@mantine/core';	
1216	
1217 interface Props {	
1218 size?: number string;	
1219 }	
1220	
1221 function IconDrawio({ size }: Props) {	
1222 return (	
1223 <svg	
1224 xmlns="http://www.w3.org/2000/svg"	
1225 viewBox="0 0 24 24"	
1226 fill="#08705"	
1227 style={{ width: rem(size), height: rem(size) }}	
1228 >	
1229 <path d="M19.69 13.419h-2.527l-2.667-4.555a1.292 1.292 0 001.035-1.28V4.16c0-.725-.576-1.312-1.302-1.312h0.771c-.726 0-1.312-.576-1.312 1.301v3.435c0 .619.426 1.152 1.034 1.28l-2.666 4.555H4.389c-.725 0-1.312-.576-1.312 1.301v3	
1230 </svg>	
1231);	
1232 }	
1233	
1234 export default IconDrawio;	
1235	
1236 // 文件: apps/client/src/components/icons/icon-excalidraw.tsx	
1237 import { rem } from '@mantine/core';	
1238	
1239 interface Props {	
1240 size?: number string;	
1241 }	
1242	
1243 function IconExcalidraw({ size }: Props) {	
1244 return (	
1245 <svg	
1246 xmlns="http://www.w3.org/2000/svg"	
1247 fill="#6965DB"	
1248 viewBox="0 0 24 24"	
1249 style={{ width: rem(size), height: rem(size) }}	
1250 >	

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 26 页
1251	<path d="M23.943 19.006a.196.196 0 0-.168-.034c-1.26-1.855-2.073-3.61-4.419-5.315l-.252-.284c-.001-.073-.067-.12-.134-.15l-.084-.084c-.05-.1-.169-.167-.286-.1-.47.234-.907.585-1.327.919-.554.434-1.109.87-1.63 1.354a5.058 5.0	
1252	</svg>	
1253	);	
1254	}	
1255		
1256	export default IconExcalidraw;	
1257		
1258	// 文件: apps/client/src/components/icons/icon-mermaid.tsx	
1259	import { rem } from "@mantine/core";	
1260		
1261	interface Props {	
1262	size?: number string;	
1263	}	
1264		
1265	function IconMermaid({ size }: Props) {	
1266	return (	
1267	<svg	
1268	xmlns="http://www.w3.org/2000/svg"	
1269	fill="#FF3670"	
1270	viewBox="0 0 24 24"	
1271	style={{ width: rem(size), height: rem(size) }}	
1272	>	
1273	<path d="M23.99 2.115A12.223 12.223 0 012 10.149 12.223 0 00.01 2.115a12.23 12.23 0 005.32 10.604 6.562 6.562 0 012.845 5.423v3.754h7.65v-3.754a6.561 6.561 0 012.844-5.423 12.223 12.223 0 005.32-10.604z"></path>	
1274	</svg>	
1275	);	
1276	}	
1277		
1278	export default IconMermaid;	
1279		
1280	// 文件: apps/client/src/components/icons/icon-people-circle.tsx	
1281	import { ActionIcon, rem } from "@mantine/core";	
1282	import React from "react";	
1283	import { IconUsersGroup } from "@table/icon-react";	
1284		
1285	export function IconGroupCircle() {	
1286	return (	
1287	<ActionIcon variant="light" size="lg" color="gray" radius="xl">	
1288	<IconUsersGroup stroke={1.5} />	
1289	</ActionIcon>	
1290	);	
1291	}	
1292		
1293	// 文件: apps/client/src/components/icons/index.ts	
1294	export { AirtableIcon } from "./airtable-icon.tsx";	
1295	export { FigmaIcon } from "./figma-icon.tsx";	
1296	export { TypeformIcon } from "./typeform-icon.tsx";	
1297	export { VimeoIcon } from "./vimeo-icon.tsx";	
1298	export { MiroIcon } from "./miro-icon.tsx";	
1299	export { GoogleDriveIcon } from "./google-drive-icon.tsx";	
1300	export { GoogleSheetsIcon } from "./google-sheets-icon.tsx";	


```
1351   size?: number | string;
1352 }
1353
1354 export function OpenIdIcon({ size }: Props) {
1355   return (
1356     <svg
1357       xmlns="http://www.w3.org/2000/svg"
1358       viewBox="0 0 24 24"
1359       fill="none"
1360       stroke="currentColor"
1361       strokeWidth="2"
1362       style={{ width: rem(size), height: rem(size) }}
1363     >
1364       <path d="M14.54,889l-3.63 1.773v18.17c-4.15-.52-7.27-2.78-7.27-5.5 0-2.58 2.8-4.75 6.63-5.41v-2.31C4.42 8.322 0 11.502 0 15.332c0 3.96 4.74 7.24 10.91 7.78l3.63-1.71V.888m.64 6.724v2.31c1.43.25 2.71.7 3.76 1.31l-1.97 1.11 7.0
1365     </svg>
1366   );
1367 }
1368
1369 // 文件: apps/client/src/components/icons/typeform-icon.tsx
1370 import { rem } from '@mantine/core';
1371
1372 interface Props {
1373   size?: number | string;
1374 }
1375
1376 export function TypeformIcon({ size }: Props) {
1377   return (
1378     <svg
1379       xmlns="http://www.w3.org/2000/svg"
1380       viewBox="0 0 24 24"
1381       style={{ width: rem(size), height: rem(size) }}
1382     >
1383       <path
1384         d="M15.502 13.035c-.5 0-.756-.411-.756-.917 0-.505.252-.894.756-.894.513 0 .756.407.756.894-.004.515-.261.917-.756.917Zm-4.888-1.81c.292 0 .414.17.414.317 0 .357-.365.514-1.126.536 0-.442.253-.854.712-.854Zm-3.241 1.81c-.47
1385     </svg>
1386   );
1387 }
1388
1389 // 文件: apps/client/src/components/icons/vimeo-icon.tsx
1390 import { rem } from '@mantine/core';
1391
1392 interface Props {
1393   size?: number | string;
1394 }
1395
1396 export function VimeoIcon({ size }: Props) {
1397   return (
1398     <svg
1399       xmlns="http://www.w3.org/2000/svg"
1400       viewBox="0 0 24 24"
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料

鲜知（北京）科技有限公司 | 第 29 页

```
1401         fill="#1AB7EA"
1402         style={{ width: rem(size), height: rem(size) }}
1403     >
1404     <path
1405       d="M23.9765 6.4168c-.105 2.338-1.739 5.5429-4.894 9.6088-3.2679 4.247-6.0258 6.3699-8.2898 6.3699-1.409 0-2.578-1.294-3.553-3.8811-1.9179-7.1138c-.719-2.584-1.488-3.878-2.312-3.878-.179 0-.806.378-1.8809 1.1321-1.129-1.457a
1406     </svg>
1407   );
1408 }
1409
1410 // 文件: apps/client/src/components/icons/youtube-icon.tsx
1411 import { rem } from '@mantine/core';
1412
1413 interface Props {
1414   size?: number | string;
1415 }
1416
1417 export function YoutubeIcon({ size }: Props) {
1418   return (
1419     <svg
1420       xmlns="http://www.w3.org/2000/svg"
1421       viewBox="0 0 24 24"
1422       fill="#F00000"
1423       style={{ width: rem(size), height: rem(size) }}
1424     >
1425       <path
1426         d="M23.498 6.186a3.016 3.016 0 0 0-2.122-2.136C19.505 3.545 12 3.545s-7.505 0-9.377.505A3.017 3.017 0 0 0 .502 6.186C0 8.07 0 12 0 12s0 3.93.502 5.814a3.016 3.016 0 0 0 2.122 2.136c1.871.505 9.376.505s9.376.505s7.5
1427     </svg>
1428   );
1429 }
1430
1431 /* 文件: apps/client/src/components/layouts/global/app-header.module.css */
1432 .header {
1433   height: 100%;
1434   margin-bottom: rem(120px);
1435   background-color: var(--ui-bg-surface);
1436   border-bottom: rem(1px) solid var(--ui-border-default);
1437   padding-left: var(--mantine-spacing-md);
1438   padding-right: var(--mantine-spacing-md);
1439 }
1440
1441 .link {
1442   display: block;
1443   line-height: 1;
1444   padding: rem(8px) rem(12px);
1445   border-radius: rem(8px);
1446   text-decoration: none;
1447   color: var(--ui-text-secondary);
1448   font-size: var(--mantine-font-size-sm);
1449   font-weight: 500;
1450   transition: background-color 120ms ease, color 120ms ease;
```

```
1451
1452     @mixin hover {
1453         background-color: var(--ui-bg-subtle);
1454         color: var(--ui-text-primary);
1455     }
1456 }
1457
1458 /* 文件: apps/client/src/components/layouts/global/app-shell.module.css */
1459 /* Header must be opaque so scrolling content does not show through (sticky best practice). */
1460 .header {
1461     background-color: var(--ui-bg-surface);
1462 }
1463
1464 .navbar {
1465     background-color: transparent; /* body is the single layer */
1466 }
1467
1468 .navbar {
1469     top: var(--app-shell-header-height, 52px) !important;
1470     height: calc(100dvh - var(--app-shell-header-height, 52px)) !important;
1471
1472     @media (max-width: $mantine-breakpoint-sm) {
1473         width: 350px;
1474     }
1475 }
1476
1477 .navbar {
1478     border-inline-end: 1px solid var(--ui-border-default);
1479 }
1480
1481
1482 .pageAsidePanel {
1483     position: fixed;
1484     inset-block-start: 90px;
1485     inset-block-end: 0;
1486     width: clamp(280px, 25vw, 420px);
1487     min-width: 280px;
1488     max-width: 420px;
1489     background-color: var(--ui-bg-surface);
1490     z-index: 210;
1491     opacity: 0;
1492     pointer-events: none;
1493     overflow: hidden;
1494     transition:
1495         transform 180ms ease,
1496         opacity 120ms ease;
1497 }
1498
1499 .openPanel {
1500     opacity: 1;
```

```
1501 import { convertTableNodeToArrayOfRows } from './convert-table-node-to-array-of-rows'
1502 import { getSelectionRangeInColumn } from './get-selection-range-in-column'
1503 import { moveRowInArrayOfRows } from './move-row-in-array-of-rows'
1504 import { findTable } from './query'
1505 import { transpose } from './transpose'
1506
1507 export interface MoveColumnParams {
1508   tr: Transaction
1509   originIndex: number
1510   targetIndex: number
1511   select: boolean
1512   pos: number
1513 }
1514
1515 /**
1516  * Move a column from index `origin` to index `target`.
1517  *
1518  * @internal
1519  */
1520 export function moveColumn(moveColParams: MoveColumnParams): boolean {
1521   const { tr, originIndex, targetIndex, select, pos } = moveColParams
1522   const $pos = tr.doc.resolve(pos)
1523   const table = findTable($pos)
1524   if (!table) return false
1525
1526   const indexesOriginColumn = getSelectionRangeInColumn(tr, originIndex)?.indexes
1527   const indexesTargetColumn = getSelectionRangeInColumn(tr, targetIndex)?.indexes
1528
1529   if (!indexesOriginColumn || !indexesTargetColumn) return false
1530
1531   if (indexesOriginColumn.includes(targetIndex)) return false
1532
1533   const newTable = moveTableColumn(
1534     table.node,
1535     indexesOriginColumn,
1536     indexesTargetColumn,
1537     0,
1538   )
1539
1540   tr.replaceWith(
1541     table.pos,
1542     table.pos + table.node.nodeSize,
1543     newTable,
1544   )
1545
1546   if (!select) return true
1547
1548   const map = TableMap.get(newTable)
1549   const start = table.start
1550   const index = targetIndex
```

```
1551   const lastCell = map.positionAt(map.height - 1, index, newTable)
1552   const $lastCell = tr.doc.resolve(start + lastCell)
1553
1554   const firstCell = map.positionAt(0, index, newTable)
1555   const $firstCell = tr.doc.resolve(start + firstCell)
1556
1557   tr.setSelection(CellSelection.colSelection($lastCell, $firstCell))
1558   return true
1559 }
1560
1561 function moveTableColumn(
1562   table: Node,
1563   indexesOrigin: number[],
1564   indexesTarget: number[],
1565   direction: -1 | 1 | 0,
1566 ) {
1567   let rows = transpose(convertTableNodeToArrayOfRows(table))
1568
1569   rows = moveRowInArrayOfRows(rows, indexesOrigin, indexesTarget, direction)
1570   rows = transpose(rows)
1571
1572   return convertArrayOfRowsToTableNode(table, rows)
1573 }
1574
1575 // 文件: packages/editor-ext/src/lib/table/utils/move-row-in-array-of-rows.ts
1576 /**
1577  * Move a row in an array of rows.
1578  *
1579  * @internal
1580  */
1581 export function moveRowInArrayOfRows<T>({
1582   rows: T[],
1583   indexesOrigin: number[],
1584   indexesTarget: number[],
1585   directionOverride: -1 | 1 | 0,
1586 }): T[] {
1587   const direction = indexesOrigin[0] > indexesTarget[0] ? -1 : 1
1588
1589   const rowsExtracted = rows.splice(indexesOrigin[0], indexesOrigin.length)
1590   const positionOffset = rowsExtracted.length % 2 === 0 ? 1 : 0
1591   let target: number
1592
1593   if (directionOverride === -1 && direction === 1) {
1594     target = indexesTarget[0] - 1
1595   } else if (directionOverride === 1 && direction === -1) {
1596     target = indexesTarget[indexesTarget.length - 1] - positionOffset + 1
1597   } else {
1598     target = direction === -1
1599       ? indexesTarget[0]
1600       : indexesTarget[indexesTarget.length - 1] - positionOffset
```

```
1601     }
1602
1603     rows.splice(target, 0, ...rowsExtracted)
1604     return rows
1605   }
1606
1607   // 文件: packages/editor-ext/src/lib/table/utils/move-row.ts
1608   import type { Node } from '@tiptap/pm/model'
1609   import type { Transaction } from '@tiptap/pm/state'
1610   import {
1611     CellSelection,
1612     TableMap,
1613   } from '@tiptap/pm/tables'
1614
1615   import { convertArrayOfRowsToTableNode } from './convert-array-of-rows-to-table-node'
1616   import { convertTableNodeToArrayOfRows } from './convert-table-node-to-array-of-rows'
1617   import { getSelectionRangeInRow } from './get-selection-range-in-row'
1618   import { moveRowInArrayOfRows } from './move-row-in-array-of-rows'
1619   import { findTable } from './query'
1620
1621   export interface MoveRowParams {
1622     tr: Transaction
1623     originIndex: number
1624     targetIndex: number
1625     select: boolean
1626     pos: number
1627   }
1628
1629   /**
1630    * Move a row from index `origin` to index `target`.
1631    *
1632    * @internal
1633    */
1634   export function moveRow(moveRowParams: MoveRowParams): boolean {
1635     const { tr, originIndex, targetIndex, select, pos } = moveRowParams
1636     const $pos = tr.doc.resolve(pos)
1637     const table = findTable($pos)
1638     if (!table) return false
1639
1640     const indexesOriginRow = getSelectionRangeInRow(tr, originIndex)?.indexes
1641     const indexesTargetRow = getSelectionRangeInRow(tr, targetIndex)?.indexes
1642
1643     if (!indexesOriginRow || !indexesTargetRow) return false
1644
1645     if (indexesOriginRow.includes(targetIndex)) return false
1646
1647     const newTable = moveTableRow(table.node, indexesOriginRow, indexesTargetRow, 0)
1648
1649     tr.replaceWith(
1650       table.pos,
```

```
1651     table.pos + table.node.nodeSize,
1652     newTable,
1653   )
1654
1655   if (!select) return true
1656
1657   const map = TableMap.get(newTable)
1658   const start = table.start
1659   const index = targetIndex
1660   const lastCell = map.positionAt(index, map.width - 1, newTable)
1661   const $lastCell = tr.doc.resolve(start + lastCell)
1662
1663   const firstCell = map.positionAt(index, 0, newTable)
1664   const $firstCell = tr.doc.resolve(start + firstCell)
1665
1666   tr.setSelection(CellSelection.rowSelection($lastCell, $firstCell))
1667   return true
1668 }
1669
1670 function moveTableRow(
1671   table: Node,
1672   indexesOrigin: number[],
1673   indexesTarget: number[],
1674   direction: -1 | 1 | 0,
1675 ) {
1676   let rows = convertTableNodeToArrayOfRows(table)
1677
1678   rows = moveRowInArrayOfRows(rows, indexesOrigin, indexesTarget, direction)
1679
1680   return convertArrayOfRowsToTableNode(table, rows)
1681 }
1682
1683 // 文件: packages/editor-ext/src/lib/table/utils/query.ts
1684 import type {
1685   Node,
1686   ResolvedPos,
1687 } from '@tiptap/pm/model'
1688 import type { Selection } from '@tiptap/pm/state'
1689 import {
1690   cellAround,
1691   cellNear,
1692   CellSelection,
1693   inSameTable,
1694 } from '@tiptap/pm/tables'
1695
1696 /**
1697  * Checks if the given object is a `CellSelection` instance.
1698  *
1699  * @public
1700  */
```

```
1701 export function isCellSelection(value: unknown): value is CellSelection {
1702   return value instanceof CellSelection
1703 }
1704
1705 /**
1706  * Find the closest table node.
1707  *
1708  * @internal
1709  */
1710 export function findTable($pos: ResolvedPos): FindParentNodeResult | undefined {
1711   return findParentNode((node) => node.type.spec.tableRole === 'table', $pos)
1712 }
1713
1714 /**
1715  * Try to find the anchor and head cell in the same table by using the given
1716  * anchor and head as hit points, or fallback to the selection's anchor and
1717  * head.
1718  *
1719  * @internal
1720  */
1721 export function findCellRange(
1722   selection: Selection,
1723   anchorHit?: number,
1724   headHit?: number,
1725 ): [ResolvedPos, ResolvedPos] | undefined {
1726   if (anchorHit == null && headHit == null && isCellSelection(selection)) {
1727     return [selection.$anchorCell, selection.$headCell]
1728   }
1729
1730   const anchor: number = anchorHit ?? headHit ?? selection.anchor
1731   const head: number = headHit ?? anchorHit ?? selection.head
1732
1733   const doc = selection.$head.doc
1734
1735   const $anchorCell = findCellPos(doc, anchor)
1736   const $headCell = findCellPos(doc, head)
1737
1738   if ($anchorCell && $headCell && inSameTable($anchorCell, $headCell)) {
1739     return [$anchorCell, $headCell]
1740   }
1741 }
1742
1743 /**
1744  * Try to find a resolved pos of a cell by using the given pos as a hit point.
1745  *
1746  * @internal
1747  */
1748 export function findCellPos(
1749   doc: Node,
1750   pos: number,
```

```
1751 ): ResolvedPos | undefined {
1752   const $pos = doc.resolve(pos)
1753   return cellAround($pos) || cellNear($pos)
1754 }
1755
1756 /**
1757  * @internal
1758  */
1759 export interface FindParentNodeResult {
1760   /**
1761    * The closest parent node that satisfies the predicate.
1762    */
1763   node: Node
1764
1765   /**
1766    * The position directly before the node.
1767    */
1768   pos: number
1769
1770   /**
1771    * The position at the start of the node.
1772    */
1773   start: number
1774
1775   /**
1776    * The depth of the node.
1777    */
1778   depth: number
1779 }
1780
1781 /**
1782  * Find the closest parent node that satisfies the predicate.
1783  *
1784  * @internal
1785  */
1786 export function findParentNode(
1787   /**
1788    * The predicate to test the parent node.
1789    */
1790   predicate: (node: Node) => boolean,
1791   /**
1792    * The position to start searching from.
1793    */
1794   $pos: ResolvedPos,
1795 ): FindParentNodeResult | undefined {
1796   for (let depth = $pos.depth; depth >= 0; depth -= 1) {
1797     const node = $pos.node(depth)
1798
1799     if (predicate(node)) {
1800       const pos = depth === 0 ? 0 : $pos.before(depth)
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 37 页
1801	const start = \$pos.start(depth)	
1802	return { node, pos, start, depth }	
1803	}	
1804	}	
1805	}	
1806		
1807	// 文件: packages/editor-ext/src/lib/table/utls/transpose.ts	
1808	/**	
1809	* Transposes a 2D array by flipping columns to rows.	
1810	*	
1811	* Transposition is a familiar algebra concept where the matrix is flipped	
1812	* along its diagonal. For more details, see:	
1813	* https://en.wikipedia.org/wiki/Transpose	
1814	*	
1815	* @example	
1816	* ```javascript	
1817	* const arr = [	
1818	* ['a1', 'a2', 'a3'],	
1819	* ['b1', 'b2', 'b3'],	
1820	* ['c1', 'c2', 'c3'],	
1821	* ['d1', 'd2', 'd3'],	
1822	*];	
1823	*	
1824	* const result = transpose(arr);	
1825	* result === [	
1826	* ['a1', 'b1', 'c1', 'd1'],	
1827	* ['a2', 'b2', 'c2', 'd2'],	
1828	* ['a3', 'b3', 'c3', 'd3'],	
1829	*]	
1830	* ```	
1831	*/	
1832	export function transpose<T>(array: T[][][]): T[][] {	
1833	return array[0].map((_, i) => {	
1834	return array.map((column) => column[i])	
1835	})	
1836	}	
1837		
1838	// 文件: packages/editor-ext/src/lib/table/utls/types.ts	
1839	import type {	
1840	Node,	
1841	ResolvedPos,	
1842	} from '@tiptap/pm/model'	
1843		
1844	export type CellPos = {	
1845	pos: number	
1846	start: number	
1847	depth: number	
1848	node: Node	
1849	}	
1850		

```
1851 export type CellSelectionRange = {
1852   $anchor: ResolvedPos
1853   $head: ResolvedPos
1854   // an array of column/row indexes
1855   indexes: number[]
1856 }
1857
1858 // 文件: packages/editor-ext/src/lib/trailing-node.ts
1859 import { Extension } from '@tiptap/core'
1860 import { PluginKey, Plugin } from '@tiptap/pm/state'
1861
1862 export interface TrailingNodeExtensionOptions {
1863   node: string,
1864   notAfter: string[],
1865 }
1866
1867 function nodeEqualsType({ types, node }: { types: any, node: any }) {
1868   return (Array.isArray(types) && types.includes(node.type)) || node.type === types
1869 }
1870
1871 // @ts-ignore
1872 /**
1873  * Extension based on:
1874  * - https://github.com/ueberdosis/tiptap/blob/v1/packages/tiptap-extensions/src/extensions/TrailingNode.js
1875  * - https://github.com/remirror/remirror/blob/e0f1bec4a1e8073ce8f5500d62193e52321155b9/packages/prosemirror-trailing-node/src/trailing-node-plugin.ts
1876  */
1877 export const TrailingNode = Extension.create<TrailingNodeExtensionOptions>({
1878   name: 'trailingNode',
1879
1880   addOptions() {
1881     return {
1882       node: 'paragraph',
1883       notAfter: [
1884         'paragraph',
1885       ],
1886     };
1887   },
1888
1889   addProseMirrorPlugins() {
1890     const plugin = new PluginKey(this.name)
1891     const disabledNodes = Object.entries(this.editor.schema.nodes)
1892       .map([, value]) => value
1893       .filter(node => this.options.notAfter.includes(node.name))
1894
1895     return [
1896       new Plugin({
1897         key: plugin,
1898         appendTransaction: (_, __, state) => {
1899           const { doc, tr, schema } = state
1900           const shouldInsertNodeAtEnd = plugin.getState(state)
```

```
1901     const endPosition = doc.content.size;
1902     const type = schema.nodes[this.options.node]
1903
1904     if (!shouldInsertNodeAtEnd) {
1905         return;
1906     }
1907
1908     return tr.insert(endPosition, type.create());
1909 },
1910 state: {
1911     init: (_, state) => {
1912         try {
1913             const lastNode = state.tr.doc.lastChild
1914             return !nodeEqualsType({ node: lastNode, types: disabledNodes })
1915         } catch (err){
1916             console.log(err)
1917         }
1918         return true;
1919     },
1920     apply: (tr, value) => {
1921         if (!tr.docChanged) {
1922             return value
1923         }
1924
1925         // Ignore transactions from UniqueID extension to prevent infinite loops
1926         // when UniqueID adds IDs to newly inserted trailing nodes
1927         if (tr.getMeta('__uniqueIDTransaction')) {
1928             return value
1929         }
1930
1931         const lastNode = tr.doc.lastChild
1932         return !nodeEqualsType({ node: lastNode, types: disabledNodes })
1933     },
1934     },
1935     },
1936     },
1937     }
1938 })
1939
1940 // 文件: packages/editor-ext/src/lib/types.ts
1941 // repetition for now
1942 export interface IAttachment {
1943     id: string;
1944     fileName: string;
1945     filePath: string;
1946     fileSize: number;
1947     fileExt: string;
1948     mimeType: string;
1949     type: string;
1950     creatorId: string;
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料	鲜知（北京）科技有限公司 第 40 页
<pre>1951 pageId: string null; 1952 spaceId: string null; 1953 workspaceId: string; 1954 createdAt: string; 1955 updatedAt: string; 1956 deletedAt: string null; 1957 } 1958 1959 // 文件: packages/editor-ext/src/lib/unique-id/index.ts 1960 export { UniqueID } from './unique-id'; 1961 export * from './unique-id.util'; 1962 1963 // 文件: packages/editor-ext/src/lib/unique-id/unique-id.ts 1964 import { generateNodeId } from './utils'; 1965 import { UniqueID as TiptapUniqueID } from '@tiptap/extension-unique-id'; 1966 1967 export const UniqueID = TiptapUniqueID.extend({ 1968 addOptions() { 1969 return { 1970 ...this.parent?()., 1971 generateID: () => generateNodeId(), 1972 }; 1973 }, 1974 }); 1975 1976 // 文件: packages/editor-ext/src/lib/unique-id/unique-id.util.ts 1977 import type { Extensions, JSONContent } from '@tiptap/core'; 1978 import { findChildren, getSchema } from '@tiptap/core'; 1979 import { Node } from '@tiptap/pm/model'; 1980 import { EditorState } from '@tiptap/pm/state'; 1981 import type { UniqueID } from './unique-id'; 1982 1983 /** 1984 * Creates a new document with unique IDs added to the nodes. Does the same 1985 * thing as the UniqueID extension, but without the need to create an `Editor` 1986 * instance. This lets you add unique IDs to the document in the server. 1987 * 1988 * When you call it, include the `UniqueID` extension in the `extensions` array. 1989 * The configuration from the `UniqueID` extension will be picked up 1990 * automatically, including its configuration options like `types` and 1991 * `attributeName`. 1992 * 1993 * @see `UniqueID` extension for more information. 1994 * 1995 * @throws {Error} If the `UniqueID` extension is not found in the extensions array. 1996 * 1997 * @example 1998 * const doc = { 1999 * type: 'doc', 2000 * content: [</pre>	

```
2801 *      { type: 'paragraph', content: [{ type: 'text', text: 'Hello, world!' }] }
2802 *    }
2803 *  }
2804 *  const newDoc = addUniqueIds(doc, [StarterKit, UniqueID.configure({ types: ['paragraph', 'heading'] })])
2805 *  console.log(newDoc)
2806 *  // Result:
2807 *  // {
2808 *    //   type: 'doc',
2809 *    //   content: [
2810 *    //     { type: 'paragraph', content: [{ type: 'text', text: 'Hello, world!' }], id: '123' }
2811 *    //   ]
2812 *    // }
2813 *
2814 *  @param doc - A Tiptap JSON document to add unique IDs to.
2815 *  @param extensions - The extensions to use. Must include the `UniqueID` extension.
2816 *  @returns The updated Tiptap JSON document, with the unique IDs added to the nodes.
2817 */
2818 export function addUniqueIdsToDoc(
2819   doc: JSONContent,
2820   extensions: Extensions,
2821 ): JSONContent {
2822   // Find the UniqueID extension in the extensions array. If it's not found, throw an error.
2823   const uniqueIDExtension = extensions.find(
2824     (ext) => ext.name === "uniqueID",
2825   ) as typeof UniqueID | undefined;
2826   if (!uniqueIDExtension) {
2827     throw new Error("UniqueID extension not found in the extensions array");
2828   }
2829   const { types, attributeName, generateID } = uniqueIDExtension.options;
2830
2831   // Convert the JSON content to a ProseMirror node
2832   const schema = getSchema([
2833     ...extensions.filter((ext) => ext.name !== "uniqueID"),
2834     uniqueIDExtension,
2835   ]);
2836   const contentNode = Node.fromJSON(schema, doc);
2837
2838   // Find nodes that don't have a unique ID
2839   const nodesWithoutId = findChildren(contentNode, (node) => {
2840     return !node.attrs[attributeName] && types.includes(node.type.name);
2841   });
2842
2843   // Edit the document to add unique IDs to the nodes that don't have a unique ID
2844   let tr = EditorState.create({
2845     doc: contentNode,
2846   }).tr;
2847   // eslint-disable-next-line no-restricted-syntax
2848   for (const { node, pos } of nodesWithoutId) {
2849     tr = tr.setNodeAttribute(pos, attributeName, generateID({ node, pos }));
2850   }
2851 }
```

```
2051
2052 // Return the updated document
2053 return tr.doc.toJSON();
2054 }
2055
2056 // 文件: packages/editor-ext/src/lib/utlis.ts
2057 import { Editor, findParentNode, isTextSelection } from "@tiptap/core";
2058 import { EditorState, Selection, Transaction } from "@tiptap/pm/state";
2059 import { EditorView } from "@tiptap/pm/view";
2060 import { CellSelection, TableMap } from "@tiptap/pm/tables";
2061 import { Node, ResolvedPos } from "@tiptap/pm/model";
2062 import { sanitizeUrl as braintreeSanitizeUrl } from "@braintree/sanitize-url";
2063 import { customAlphabet } from "nanoid";
2064
2065 export const isRectSelected = (rect: any) => (selection: CellSelection) => {
2066   const map = TableMap.get(selection.anchorCell.node(-1));
2067   const start = selection.anchorCell.start(-1);
2068   const cells = map.cellsInRect(rect);
2069   const selectedCells = map.cellsInRect(
2070     map.rectBetween(
2071       selection.anchorCell.pos - start,
2072       selection.headCell.pos - start,
2073     ),
2074   );
2075
2076   for (let i = 0, count = cells.length; i < count; i += 1) {
2077     if (selectedCells.indexOf(cells[i]) === -1) {
2078       return false;
2079     }
2080   }
2081   return true;
2082 };
2083
2084 export const findTable = (selection: Selection) =>
2085   findParentNode(
2086     (node) => node.type.spec.tableRole && node.type.spec.tableRole === "table",
2087   )(selection);
2088
2089 export const isCellSelection = (selection: any) =>
2090   selection instanceof CellSelection;
2091
2092 export const isColumnSelected = (columnIndex: number) => (selection: any) => {
2093   if (isCellSelection(selection)) {
2094     const map = TableMap.get(selection.anchorCell.node(-1));
2095
2096     return isRectSelected({
2097       left: columnIndex,
2098       right: columnIndex + 1,
2099       top: 0,
2100
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料

鲜知（北京）科技有限公司 | 第 43 页

2101 bottom: map.height,
2102 })(selection);
2103 }
2104
2105 return false;
2106 });
2107
2108 export const isRowSelected = (rowIndex: number) => (selection: any) => {
2109 if (isCellSelection(selection)) {
2110 const map = TableMap.get(selection.\$anchorCell.node(-1));
2111
2112 return isRectSelected({
2113 left: 0,
2114 right: map.width,
2115 top: rowIndex,
2116 bottom: rowIndex + 1,
2117 })(selection);
2118 }
2119
2120 return false;
2121 });
2122
2123 export const isTableSelected = (selection: any) => {
2124 if (isCellSelection(selection)) {
2125 const map = TableMap.get(selection.\$anchorCell.node(-1));
2126
2127 return isRectSelected({
2128 left: 0,
2129 right: map.width,
2130 top: 0,
2131 bottom: map.height,
2132 })(selection);
2133 }
2134
2135 return false;
2136 });
2137
2138 export const getCellsInColumn =
2139 (columnIndex: number | number[]) => (selection: Selection) => {
2140 const table = findTable(selection);
2141 if (table) {
2142 const map = TableMap.get(table.node);
2143 const indexes = Array.isArray(columnIndex)
2144 ? columnIndex
2145 : Array.from([columnIndex]);
2146
2147 return indexes.reduce(
2148 (acc, index) => {
2149 if (index >= 0 && index <= map.width - 1) {
2150 const cells = map.cellsInRect({

file:///root/coderepository/docmost/docs/software-copyright/20260427_superchat_v1_0/generated/SuperChat_%E7%9F%A5%...

43/60

```
2151         left: index,
2152         right: index + 1,
2153         top: 0,
2154         bottom: map.height,
2155     });
2156
2157     return acc.concat(
2158       cells.map((nodePos) => {
2159         const node = table.node.nodeAt(nodePos);
2160         const pos = nodePos + table.start;
2161
2162         return { pos, start: pos + 1, node };
2163       }),
2164     );
2165   }
2166
2167   return acc;
2168 },
2169 [] as { pos: number; start: number; node: Node | null | undefined }[],
2170 );
2171 }
2172 return null;
2173 };
2174
2175 export const getCellsInRow =
2176 (rowIndex: number | number[]) => (selection: Selection) => {
2177   const table = findTable(selection);
2178
2179   if (table) {
2180     const map = TableMap.get(table.node);
2181     const indexes = Array.isArray(rowIndex)
2182       ? rowIndex
2183       : Array.from([rowIndex]);
2184
2185     return indexes.reduce(
2186       (acc, index) => {
2187         if (index >= 0 && index <= map.height - 1) {
2188           const cells = map.cellsInRect({
2189             left: 0,
2190             right: map.width,
2191             top: index,
2192             bottom: index + 1,
2193           });
2194
2195           return acc.concat(
2196             cells.map((nodePos) => {
2197               const node = table.node.nodeAt(nodePos);
2198               const pos = nodePos + table.start;
2199               return { pos, start: pos + 1, node };
2200             })
2201           );
2202         }
2203       },
2204       [] as { pos: number; start: number; node: Node | null | undefined }[]
2205     );
2206   }
2207 }
```

```
2201         });
2202     }
2203
2204     return acc;
2205 },
2206 [] as { pos: number; start: number; node: Node | null | undefined }[],
2207 );
2208 }
2209
2210 return null;
2211 };
2212
2213 export const getCellsInTable = (selection: Selection) => {
2214     const table = findTable(selection);
2215
2216     if (table) {
2217         const map = TableMap.get(table.node);
2218         const cells = map.cellsInRect({
2219             left: 0,
2220             right: map.width,
2221             top: 0,
2222             bottom: map.height,
2223         });
2224
2225         return cells.map((nodePos) => {
2226             const node = table.node.nodeAt(nodePos);
2227             const pos = nodePos + table.start;
2228
2229             return { pos, start: pos + 1, node };
2230         });
2231     }
2232
2233     return null;
2234 };
2235
2236 export const findParentNodeClosestToPos = (
2237     $pos: ResolvedPos,
2238     predicate: (node: Node) => boolean,
2239 ) => {
2240     for (let i = $pos.depth; i > 0; i -= 1) {
2241         const node = $pos.node(i);
2242
2243         if (predicate(node)) {
2244             return {
2245                 pos: i > 0 ? $pos.before(i) : 0,
2246                 start: $pos.start(i),
2247                 depth: i,
2248                 node,
2249             };
2250         }
2251     }
2252 }
```

```
2251   }
2252
2253   return null;
2254 };
2255
2256 export const findCellClosestToPos = ($pos: ResolvedPos) => {
2257   const predicate = (node: Node) =>
2258     node.type.spec.tableRole && /cell/i.test(node.type.spec.tableRole);
2259
2260   return findParentNodeClosestToPos($pos, predicate);
2261 };
2262
2263 const select =
2264   (type: "row" | "column") => (index: number) => (tr: Transaction) => {
2265     const table = findTable(tr.selection);
2266     const isRowSelection = type === "row";
2267
2268     if (table) {
2269       const map = TableMap.get(table.node);
2270
2271       // Check if the index is valid
2272       if (index >= 0 && index < (isRowSelection ? map.height : map.width)) {
2273         const left = isRowSelection ? 0 : index;
2274         const top = isRowSelection ? index : 0;
2275         const right = isRowSelection ? map.width : index + 1;
2276         const bottom = isRowSelection ? index + 1 : map.height;
2277
2278         const cellsInFirstRow = map.cellsInRect({
2279           left,
2280           top,
2281           right: isRowSelection ? right : left + 1,
2282           bottom: isRowSelection ? top + 1 : bottom,
2283         });
2284
2285         const cellsInLastRow =
2286           bottom - top === 1
2287             ? cellsInFirstRow
2288             : map.cellsInRect({
2289               left: isRowSelection ? left : right - 1,
2290               top: isRowSelection ? bottom - 1 : top,
2291               right,
2292               bottom,
2293             });
2294
2295         const head = table.start + cellsInFirstRow[0];
2296         const anchor = table.start + cellsInLastRow[cellsInLastRow.length - 1];
2297         const $head = tr.doc.resolve(head);
2298         const $anchor = tr.doc.resolve(anchor);
2299
2300         // @ts-ignore
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料	鲜知（北京）科技有限公司 第 47 页
2301 return tr.setSelection(new CellSelection(\$anchor, \$head)); 2302 } 2303 } 2304 return tr; 2305 }; 2306 export const selectColumn = select("column"); 2307 export const selectRow = select("row"); 2308 export const selectTable = (tr: Transaction) => { 2309 const table = findTable(tr.selection); 2310 if (table) { 2311 const { map } = TableMap.get(table.node); 2312 if (map && map.length) { 2313 const head = table.start + map[0]; 2314 const anchor = table.start + map[map.length - 1]; 2315 const \$head = tr.doc.resolve(head); 2316 const \$anchor = tr.doc.resolve(anchor); 2317 // @ts-ignore 2318 return tr.setSelection(new CellSelection(\$anchor, \$head)); 2319 } 2320 } 2321 return tr; 2322 }; 2323 export const isColumnGripSelected = ({ 2324 editor, 2325 view, 2326 state, 2327 from, 2328 }): { 2329 editor: Editor; 2330 view: EditorView; 2331 state: EditorState; 2332 from: number; 2333 }) => { 2334 const domAtPos = view.domAtPos(from).node as HTMLElement; 2335 const nodeDOM = view.nodeDOM(from) as HTMLElement; 2336 const node = nodeDOM domAtPos; 2337 if (!editor.isActive("table") !node isTableSelected(state.selection)) { 2338 return false; 2339 } 2340 let container = node;	

```
2351
2352   while (container && ![ "TD", "TH"].includes(container.tagName)) {
2353     container = container.parentElement!;
2354   }
2355
2356   const gripColumn =
2357     container &&
2358     container.querySelector &&
2359     container.querySelector("a.grip-column.selected");
2360
2361   return !!gripColumn;
2362 };
2363
2364 export const isRowGripSelected = ({
2365   editor,
2366   view,
2367   state,
2368   from,
2369 }): {
2370   editor: Editor;
2371   view: EditorView;
2372   state: EditorState;
2373   from: number;
2374 }) => {
2375   const domAtPos = view.domAtPos(from).node as HTMLElement;
2376   const nodeDOM = view.nodeDOM(from) as HTMLElement;
2377   const node = nodeDOM || domAtPos;
2378
2379   if (!editor.isActive("table") || !node || isTableSelected(state.selection)) {
2380     return false;
2381   }
2382
2383   let container = node;
2384
2385   while (container && ![ "TD", "TH"].includes(container.tagName)) {
2386     container = container.parentElement!;
2387   }
2388
2389   const gripRow =
2390     container &&
2391     container.querySelector &&
2392     container.querySelector("a.grip-row.selected");
2393
2394   return !!gripRow;
2395 };
2396
2397 export function isTextSelected(editor: Editor) {
2398   const {
2399     state: {
2400       doc,
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 49 页
2401	selection,	
2402	selection: { empty, from, to },	
2403	},	
2404	} = editor;	
2405		
2406	const isEmptyTextBlock =	
2407	!doc.textBetween(from, to).length && isTextSelection(selection);	
2408		
2409	if (empty isEmptyTextBlock !editor.isEditable) {	
2410	return false;	
2411	}	
2412		
2413	return true;	
2414	}	
2415		
2416	export function setAttributes(	
2417	editor: Editor,	
2418	getPos: (() => number) boolean,	
2419	attrs: Record<string, any>,	
2420	) {	
2421	if (editor.isEditable && typeof getPos === "function") {	
2422	editor.view.dispatch(	
2423	editor.view.state.tr.setNodeMarkup(getPos(), undefined, attrs),	
2424	);	
2425	}	
2426	}	
2427		
2428	export function icon(name: string) {	
2429	return `<span class="ProseMirror-icon ProseMirror-icon-\${name}"></span>`;	
2430	}	
2431		
2432	export function sanitizeUrl(url: string undefined): string {	
2433	if (!url) return "";	
2434		
2435	const sanitized = braintreeSanitizeUrl(url);	
2436		
2437	// Return empty string instead of "about:blank"	
2438	return sanitized === "about:blank" ? "" : sanitized;	
2439	}	
2440		
2441	const alphabet = "abcdefghijklmnopqrstuvwxyz";	
2442	export const generateNodeId = customAlphabet(alphabet, 12);	
2443		
2444	export function copyToClipboard(text: string): void {	
2445	if ("clipboard" in navigator) {	
2446	navigator.clipboard.writeText(text).catch(() => {	
2447	execCommandCopy(text);	
2448	});	
2449	} else {	
2450	execCommandCopy(text);	

```
2451   }
2452 }
2453
2454 export function execCommandCopy(text: string): void {
2455   const textarea = document.createElement("textarea");
2456   textarea.value = text;
2457   textarea.style.position = "fixed";
2458   textarea.style.left = "-9999px";
2459   textarea.style.top = "-9999px";
2460   document.body.appendChild(textarea);
2461   textarea.select();
2462   document.execCommand("copy");
2463   document.body.removeChild(textarea);
2464 }
2465
2466 // 文件: packages/editor-ext/src/lib/video/index.ts
2467 export { TiptapVideo } from "../video";
2468 export * from "../video-upload";
2469
2470 // 文件: packages/editor-ext/src/lib/video/video-upload.ts
2471 import { MediaUploadOptions, UploadFn } from "../media-utils";
2472 import { IAttachment } from "../types";
2473 import { generateNodeId } from "../utils";
2474 import { Node } from "@tiptap/pm/model";
2475 import { Command } from "@tiptap/core";
2476
2477 const findVideoNodeByPlaceholderId = (
2478   doc: Node,
2479   placeholderId: string,
2480 ): { node: Node; pos: number } | null => {
2481   let result: { node: Node; pos: number } | null = null;
2482
2483   doc.descendants((node, pos) => {
2484     if (result) return false;
2485
2486     if (
2487       node.type.name === "video" &&
2488       node.attrs.placeholder?.id === placeholderId
2489     ) {
2490       result = { node, pos };
2491       return false;
2492     }
2493
2494     return true;
2495   });
2496
2497   return result;
2498 };
2499 const getVideoDimensions = (
2500   url: string,
```

```
2501 ): Promise<
2502 { width: number; height: number; aspectRatio: number } | undefined
2503 > => {
2504   return new Promise<
2505     { width: number; height: number; aspectRatio: number } | undefined
2506   >((resolve) => {
2507     const video = document.createElement("video");
2508
2509     video.preload = "metadata";
2510     video.onloadedmetadata = () => {
2511       const width = video.videoWidth;
2512       const height = video.videoHeight;
2513       const aspectRatio = height > 0 ? width / height : 1;
2514
2515       resolve({ width, height, aspectRatio });
2516     };
2517     video.onerror = () => {
2518       resolve(undefined);
2519     };
2520     video.src = url;
2521   });
2522 };
2523 const handleVideoUpload =
2524 ({ validateFn, onUpload }: MediaUploadOptions): UploadFn =>
2525 async (file, editor, pos, pageId) => {
2526   // check if the file is valid
2527   const validated = validateFn?.(file);
2528   // @ts-ignore
2529   if (!validated) return;
2530
2531   const objectUrl = URL.createObjectURL(file);
2532   const videoDimensions = await getVideoDimensions(objectUrl);
2533   const placeholderId = generateNodeId();
2534   const width = videoDimensions?.width ?? undefined;
2535   const height = videoDimensions?.height ?? undefined;
2536   const aspectRatio = videoDimensions?.aspectRatio;
2537
2538   let placeholderInserted = false;
2539
2540   editor.storage.shared.videoPreviews =
2541     editor.storage.shared.videoPreviews || [];
2542   editor.storage.shared.videoPreviews[placeholderId] = objectUrl;
2543
2544   const insertPlaceholder = (): Command => {
2545     return ({ tr, state }) => {
2546       const initialPlaceholderNode = state.schema.nodes.video?.create({
2547         placeholder: {
2548           id: placeholderId,
2549           name: file.name,
2550         },
```

```
2551         width,  
2552         height,  
2553         aspectRatio,  
2554     });  
2555  
2556     if (!initialPlaceholderNode) return false;  
2557  
2558     const { parent } = tr.doc.resolve(pos);  
2559     const isEmptyTextBlock = parent.isTextBlock && !parent.childCount;  
2560  
2561     if (isEmptyTextBlock) {  
2562         // Replace e.g. empty paragraph with the video  
2563         tr.replaceRangeWith(pos - 1, pos + 1, initialPlaceholderNode);  
2564     } else {  
2565         tr.insert(pos, initialPlaceholderNode);  
2566     }  
2567  
2568     return true;  
2569 };  
2570  
2571 const replacePlaceholderWithVideo = (attachment: IAttachment): Command => {  
2572     return ({ tr }) => {  
2573         const { pos: currentPos = null } =  
2574             findVideoNodeByPlaceholderId(tr.doc, placeholderId) || {};  
2575  
2576         // If the placeholder is not found or attachment is missing, abort the process  
2577         if (currentPos === null || !attachment) return;  
2578  
2579         // Update the placeholder node with the actual video data  
2580         tr.setNodeMarkup(currentPos, undefined, {  
2581             src: `/api/files/${attachment.id}/${attachment.fileName}`,  
2582             attachmentId: attachment.id,  
2583             title: attachment.fileName,  
2584             size: attachment.fileSize,  
2585             width,  
2586             height,  
2587             aspectRatio,  
2588         });  
2589  
2590         return true;  
2591     };  
2592 };  
2593 const removePlaceholder = (): Command => {  
2594     return ({ tr }) => {  
2595         const { pos: currentPos = null } =  
2596             findVideoNodeByPlaceholderId(tr.doc, placeholderId) || {};  
2597  
2598         if (currentPos === null) return false;  
2599  
2600         tr.delete(currentPos, currentPos + 2);
```

```
2601         return true;
2602     };
2603 };
2604
2605 // Only show the placeholder if the upload takes more than 250ms
2606 const insertPlaceholderTimeout = setTimeout(() => {
2607     editor.commands.command(insertPlaceholder());
2608     placeholderInserted = true;
2609 }, 250);
2610 const disposePreviewFile = () => {
2611     URL.revokeObjectURL(objectUrl);
2612
2613     if (editor.storage.shared.videoPreviews) {
2614         delete editor.storage.shared.videoPreviews[placeholderId];
2615     }
2616 };
2617
2618 try {
2619     const attachment: IAttachment = await onUpload(file, pageId);
2620
2621     clearTimeout(insertPlaceholderTimeout);
2622
2623     if (placeholderInserted) {
2624         setTimeout(() => {
2625             editor.commands.command(replacePlaceholderWithVideo(attachment));
2626             disposePreviewFile();
2627         }, 100);
2628     } else {
2629         editor
2630             .chain()
2631             .command(insertPlaceholder())
2632             .command(replacePlaceholderWithVideo(attachment))
2633             .run();
2634         disposePreviewFile();
2635     }
2636 } catch (error) {
2637     clearTimeout(insertPlaceholderTimeout);
2638
2639     editor.commands.command(removePlaceholder());
2640     disposePreviewFile();
2641 }
2642
2643 };
2644
2645 export { handleVideoUpload };
2646
2647 // 文件: packages/editor-ext/src/lib/video/video.ts
2648 import { ReactNodeViewRenderer } from "@tiptap/react";
2649 import { Range, Node, mergeAttributes, ResizableNodeView } from "@tiptap/core";
2650 import { normalizeFileUrl } from "../media-utils";
```

```
2651 import type { ResizableNodeViewDirection } from "@tiptap/core";
2652
2653 export type VideoResizeOptions = {
2654   enabled: boolean;
2655   directions?: ResizableNodeViewDirection[];
2656   minWidth?: number;
2657   minHeight?: number;
2658   alwaysPreserveAspectRatio?: boolean;
2659   createCustomHandle?: (direction: ResizableNodeViewDirection) => HTMLElement;
2660   className?: {
2661     container?: string;
2662     wrapper?: string;
2663     handle?: string;
2664     resizing?: string;
2665   };
2666 };
2667
2668 export interface VideoOptions {
2669   view: any;
2670   HTMLAttributes: Record<string, any>;
2671   resize: VideoResizeOptions | false;
2672 }
2673
2674 export interface VideoAttributes {
2675   src?: string;
2676   align?: string;
2677   attachmentId?: string;
2678   size?: number;
2679   width?: number | string;
2680   height?: number;
2681   aspectRatio?: number;
2682   placeholder?: {
2683     id: string;
2684     name: string;
2685   };
2686 }
2687
2688 declare module "@tiptap/core" {
2689   interface Commands<ReturnType> {
2690     videoBlock: {
2691       setVideo: (attributes: VideoAttributes) => ReturnType;
2692       setVideoAt: {
2693         attributes: VideoAttributes & { pos: number | Range }
2694       } => ReturnType;
2695       setVideoAlign: (align: "left" | "center" | "right") => ReturnType;
2696       setVideoWidth: (width: number) => ReturnType;
2697       setVideoSize: (width: number, height: number) => ReturnType;
2698     };
2699   }
2700 }
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 55 页
2701		
2702	export const TiptapVideo = Node.create<VideoOptions>({	
2703	name: "video",	
2704		
2705	group: "block",	
2706	isolating: true,	
2707	atom: true,	
2708	defining: true,	
2709	draggable: true,	
2710		
2711	addOptions() {	
2712	return {	
2713	view: null,	
2714	HTMLAttributes: {},	
2715	resize: false,	
2716	};	
2717	},	
2718		
2719	addAttributes() {	
2720	return {	
2721	src: {	
2722	default: "",	
2723	parseHTML: (element) => element.getAttribute("src"),	
2724	renderHTML: (attributes) => ({	
2725	src: attributes.src,	
2726	}),	
2727	},	
2728	attachmentId: {	
2729	default: undefined,	
2730	parseHTML: (element) => element.getAttribute("data-attachment-id"),	
2731	renderHTML: (attributes: VideoAttributes) => ({	
2732	"data-attachment-id": attributes.attachmentId,	
2733	}),	
2734	},	
2735	width: {	
2736	default: null,	
2737	parseHTML: (element) => {	
2738	const raw = element.getAttribute("width");	
2739	if (!raw) return null;	
2740	if (raw.endsWith("%")) return raw;	
2741	const num = parseFloat(raw);	
2742	return isNaN(num) ? null : num;	
2743	},	
2744	renderHTML: (attributes: VideoAttributes) => ({	
2745	width: attributes.width,	
2746	}),	
2747	},	
2748	height: {	
2749	default: null,	
2750	parseHTML: (element) => {	

```
2751     const raw = element.getAttribute("height");
2752     if (!raw) return null;
2753     const num = parseFloat(raw);
2754     return isNaN(num) ? null : num;
2755   },
2756   renderHTML: (attributes: VideoAttributes) => ({
2757     height: attributes.height,
2758   }),
2759 },
2760 size: {
2761   default: null,
2762   parseHTML: (element) => element.getAttribute("data-size"),
2763   renderHTML: (attributes: VideoAttributes) => ({
2764     "data-size": attributes.size,
2765   }),
2766 },
2767 align: {
2768   default: "center",
2769   parseHTML: (element) => element.getAttribute("data-align"),
2770   renderHTML: (attributes: VideoAttributes) => ({
2771     "data-align": attributes.align,
2772   }),
2773 },
2774 aspectRatio: {
2775   default: null,
2776   parseHTML: (element) => element.getAttribute("data-aspect-ratio"),
2777   renderHTML: (attributes: VideoAttributes) => ({
2778     "data-aspect-ratio": attributes.aspectRatio,
2779   }),
2780 },
2781 placeholder: {
2782   default: null,
2783   rendered: false,
2784 },
2785 },
2786 },
2787
2788 parseHTML() {
2789   return [
2790     {
2791       tag: "video",
2792     },
2793   ];
2794 },
2795
2796 renderHTML({ HTMLAttributes }) {
2797   return [
2798     "video",
2799     { controls: "true", ...HTMLAttributes },
2800     ["source", HTMLAttributes],
```

```
2801     };
2802   },
2803
2804   addCommands() {
2805     return {
2806       setVideo:
2807         (attrs: VideoAttributes) =>
2808           ({ commands }) => {
2809             return commands.insertContent({
2810               type: "video",
2811               attrs: attrs,
2812             });
2813           },
2814
2815       setVideoAlign:
2816         (align) =>
2817           ({ commands }) =>
2818             commands.updateAttributes("video", { align }),
2819
2820       setVideoWidth:
2821         (width) =>
2822           ({ commands }) =>
2823             commands.updateAttributes("video", {
2824               width: `${Math.max(0, Math.min(100, width))}%`,
2825             }),
2826
2827       setVideoSize:
2828         (width, height) =>
2829           ({ commands }) =>
2830             commands.updateAttributes("video", { width, height }),
2831     };
2832   },
2833
2834   addNodeView() {
2835     const resize = this.options.resize;
2836
2837     if (!resize || !resize.enabled) {
2838       this.editor.isInitialized = true;
2839       return ReactNodeViewRenderer(this.options.view);
2840     }
2841
2842     const {
2843       directions,
2844       minWidth,
2845       minHeight,
2846       alwaysPreserveAspectRatio,
2847       createCustomHandle,
2848       className,
2849     } = resize;
2850
```

```
2851 return (props) => {
2852   const { node, getPos, HTMLAttributes, editor } = props;
2853
2854   if (!node.attrs.src) {
2855     editor.isInitialized = true;
2856     const reactView = ReactNodeViewRenderer(this.options.view);
2857     const view = reactView(props);
2858
2859     const originalUpdate = view.update?.bind(view);
2860     view.update = (updatedNode, decorations, innerDecorations) => {
2861       if (updatedNode.attrs.src && !node.attrs.src) {
2862         return false;
2863       }
2864       if (originalUpdate) {
2865         return originalUpdate(updatedNode, decorations, innerDecorations);
2866       }
2867       return true;
2868     };
2869     return view;
2870   }
2871 }
2872
2873 const el = document.createElement("video");
2874 el.src = normalizeFileUrl(node.attrs.src);
2875 el.controls = true;
2876 el.preload = "metadata";
2877 el.style.display = "block";
2878 el.style.maxWidth = "100%";
2879 el.style.borderRadius = "8px";
2880
2881 if (typeof node.attrs.width === "number" && node.attrs.width > 0) {
2882   el.style.width = `${node.attrs.width}px`;
2883   if (typeof node.attrs.height === "number" && node.attrs.height > 0) {
2884     el.style.height = `${node.attrs.height}px`;
2885   }
2886 }
2887
2888 let currentNode = node;
2889
2890 const nodeView = new ResizableNodeView({
2891   element: el,
2892   editor,
2893   node,
2894   getPos,
2895   onResize: (w, h) => {
2896     el.style.width = `${w}px`;
2897     el.style.height = `${h}px`;
2898   },
2899   onCommit: () => {
2900     const pos = getPos();
```

SuperChat 知识协作管理系统 V1.0 程序鉴别材料		鲜知（北京）科技有限公司 第 59 页
2901	if (pos === undefined) return;	
2902		
2903	this.editor	
2904	.chain()	
2905	.setNodeSelection(pos)	
2906	.updateAttributes(this.name, {	
2907	width: Math.round(el.offsetWidth),	
2908	height: Math.round(el.offsetHeight),	
2909	})	
2910	.run();	
2911	},	
2912	onUpdate: (updatedNode, _decorations, _innerDecorations) => {	
2913	if (updatedNode.type !== currentNode.type) {	
2914	return false;	
2915	}	
2916		
2917	if (updatedNode.attrs.src !== currentNode.attrs.src) {	
2918	el.src = normalizeFileUrl(updatedNode.attrs.src);	
2919	}	
2920		
2921	const w = updatedNode.attrs.width;	
2922	const h = updatedNode.attrs.height;	
2923	if (w !== null) {	
2924	el.style.width = `\${w}px`;	
2925	}	
2926	if (h !== null) {	
2927	el.style.height = `\${h}px`;	
2928	}	
2929		
2930	const align = updatedNode.attrs.align "center";	
2931	const container = nodeView.dom as HTMLElement;	
2932	applyAlignment(container, align);	
2933		
2934	currentNode = updatedNode;	
2935	return true;	
2936	},	
2937	options: {	
2938	directions,	
2939	min: {	
2940	width: minWidth,	
2941	height: minHeight,	
2942	},	
2943	preserveAspectRatio: alwaysPreserveAspectRatio === true,	
2944	createCustomHandle,	
2945	className,	
2946	},	
2947	});	
2948		
2949	const dom = nodeView.dom as HTMLElement;	
2950		

```
2951     applyAlignment(dom, node.attrs.align || "center");
2952
2953     // Handle percentage width backward compat
2954     const widthAttr = node.attrs.width;
2955     if (typeof widthAttr === "string" && widthAttr.endsWith("%")) {
2956       requestAnimationFrame(() => {
2957         const parentEl = dom.parentElement;
2958         if (parentEl) {
2959           const containerWidth = parentEl.clientWidth;
2960           const pctValue = parseInt(widthAttr, 10);
2961           if (!isNaN(pctValue) && containerWidth > 0) {
2962             const pxWidth = Math.round(
2963               containerWidth * (pctValue / 100),
2964             );
2965             el.style.width = `${pxWidth}px`;
2966             if (node.attrs.aspectRatio) {
2967               el.style.height = `${Math.round(pxWidth / node.attrs.aspectRatio)}px`;
2968             }
2969           }
2970         }
2971         dom.style.visibility = "";
2972         dom.style.pointerEvents = "";
2973       });
2974     }
2975
2976     // Show skeleton background while video loads from server
2977     dom.style.pointerEvents = "none";
2978     dom.style.background =
2979       "light-dark(var(--mantine-color-gray-2), var(--mantine-color-dark-6))";
2980
2981     el.onloadedmetadata = () => {
2982       dom.style.pointerEvents = "";
2983       dom.style.background = "";
2984     };
2985
2986     return nodeView;
2987   };
2988 },
2989 });
2990
2991 function applyAlignment(container: HTMLElement, align: string) {
2992   if (align === "left") {
2993     container.style.justifyContent = "flex-start";
2994   } else if (align === "right") {
2995     container.style.justifyContent = "flex-end";
2996   } else {
2997     container.style.justifyContent = "center";
2998   }
2999 }
3000 }
```